

Section-Based – Security (CDA)

1. 1. Question

Category: CDA – Security

A company is using AWS Organizations to manage its multiple AWS accounts which is being used by its various departments. To avoid security issues, it is of utmost importance to test the impact of service control policies (SCPs) on your IAM policies and resource policies before applying them.

Which of the following services can you use to test and troubleshoot IAM and resource-based policies?

- Amazon Inspector
- AWS Config
- IAM Policy Simulator
- Systems Manager

Incorrect

The IAM policy simulator evaluates the policies that you choose and determines the effective permissions for each of the actions that you specify. The simulator uses the same policy evaluation engine that is used during real requests to AWS services. But the simulator differs from the live AWS environment in the following ways:

- The simulator does not make an actual AWS service request, so you can safely test requests that might make unwanted changes to your live AWS environment.
- Because the simulator does not simulate running the selected actions, it cannot report any response to the simulated request. The only result returned is whether the requested action would be allowed or denied.
- If you edit a policy inside the simulator, these changes affect only the simulator. The corresponding policy in your AWS account remains unchanged.

Check out this video on IAM Policy Simulator:

With the IAM policy simulator, you can test and troubleshoot IAM and resource-based policies in the following ways:

- Test policies that are attached to IAM users, groups, or roles in your AWS account. If more than one policy is attached to the user, group, or role, you can test all the policies, or select individual policies to test. You can test which actions are allowed or denied by the selected policies for specific resources.
- Test policies that are attached to AWS resources, such as Amazon S3 buckets, Amazon SQS queues, Amazon SNS topics, or Amazon S3 Glacier vaults.
- If your AWS account is a member of an organization in [AWS Organizations](#), then you can test the impact of service control policies (SCPs) on your IAM policies and resource policies.
- Test new policies that are not yet attached to a user, group, or role by typing or copying them into the simulator. These are used only in the simulation and are not saved. Take note that you cannot type or copy a resource-based policy into the simulator. To use a resource-based policy in the simulator, you must include the resource in the simulation and select the checkbox to include that resource's policy in the simulation.
- Test the policies with selected services, actions, and resources. For example, you can test to ensure that your policy allows an entity to perform the `ListAllMyBuckets`, `CreateBucket`, and `DeleteBucket` actions in the Amazon S3 service on a specific bucket.
- Simulate real-world scenarios by providing context keys, such as an IP address or date, that are included in `Condition` elements in the policies being tested.
- Identify which specific statement in a policy results in allowing or denying access to a particular resource or action.

Hence, the correct answer is **IAM Policy Simulator**.

AWS Config is incorrect because this is just a service that enables you to assess, audit, and evaluate the configurations of your AWS resources.

Systems Manager is incorrect because this service just provides a unified user interface so you can view operational data from multiple AWS services and allows you to automate operational tasks across your AWS resources. Unlike IAM Policy Simulator, it can't be used to simulate your policies.

Amazon Inspector is incorrect because it is just an automated security assessment service that helps improve the security and compliance of applications deployed on AWS.

References:

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_testing-policies.html

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_examples_iam_policy-sim-path-console.html

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

2. 2. Question

Category: CDA – Security

A developer is using API Gateway Lambda Authorizer to provide authentication for every API request and control access to your API. The requirement is to implement an authentication strategy which is similar to OAuth or SAML.

Which of the following is the MOST suitable method that the developer should use in this scenario?

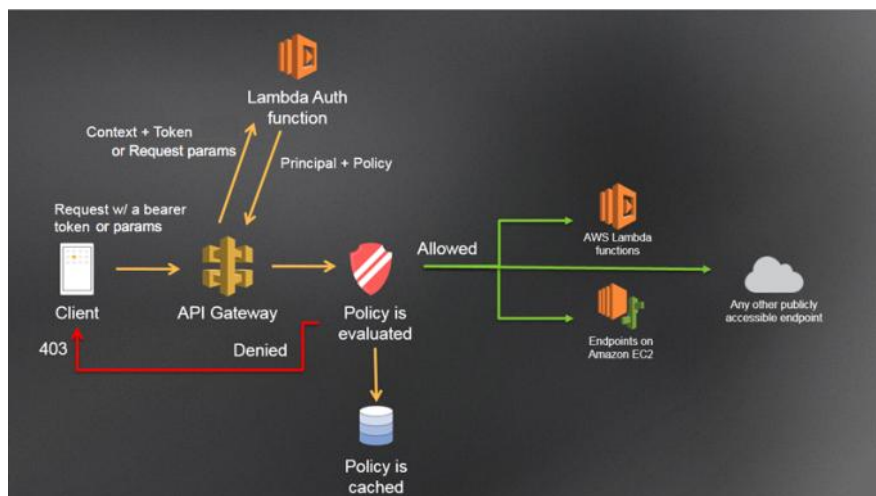
- Request Parameter-based Authorization
- Cross-Account Lambda Authorizer
- Token-based Authorization
- AWS STS-based Authentication

Incorrect

A Lambda authorizer is an API Gateway feature that uses a Lambda function to control access to your API. When a client makes a request to one of your API's methods, API Gateway calls your Lambda authorizer, which takes the caller's identity as input and returns an IAM policy as output.

There are two types of Lambda authorizers:

- A **token-based** Lambda authorizer (also called a **TOKEN** authorizer) receives the caller's identity in a bearer token, such as a JSON Web Token (JWT) or an OAuth token.
- A **request parameter-based** Lambda authorizer (also called a **REQUEST** authorizer) receives the caller's identity in a combination of headers, query string parameters, stageVariables, and \$context variables.



It is possible to use an AWS Lambda function from an AWS account that is different from the one in which you created your Lambda authorizer function by using a Cross-Account Lambda Authorizer.

Therefore, the correct answer in this scenario is to use a **token-based authorization** since this is useful if you want to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML.

Requesting Parameter-based Lambda Authorization is incorrect because this does not use tokens to identify a caller but through a combination of headers, query string parameters, stageVariables, and \$context variables.

AWS STS-based authentication is incorrect because this is not a valid type of API Gateway Lambda Authorizer.

Cross-Account Lambda Authorizer is incorrect because this just enables you to use an AWS Lambda function from a different AWS account as your API authorizer function. Moreover, this is not a valid Lambda authorizer type.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-lambda-authorizer-input.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-lambda-authorizer-cross-account-lambda-authorizer.html>

Check out this Amazon API Gateway Cheat Sheet:
<https://tutorialsdojo.com/amazon-api-gateway/>

3. 3. Question

Category: CDA – Security

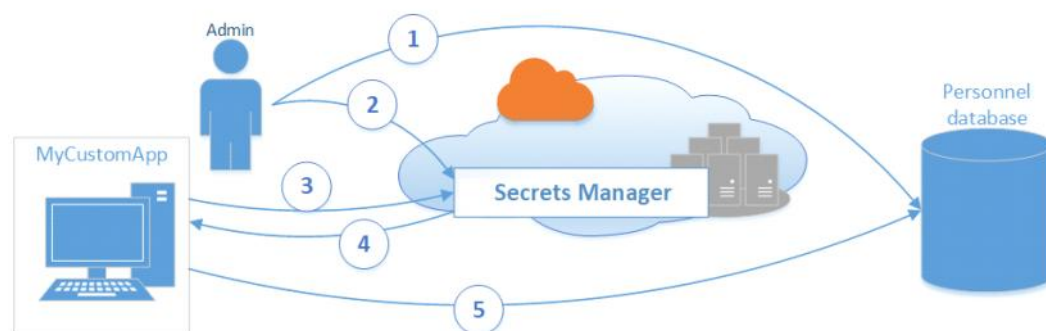
A company has a suite of web applications that is heavily using RDS database in Multi-AZ Deployments configuration with several Read Replicas. For improved security, you were instructed to ensure that all of their database credentials, API keys, and other secrets are encrypted and rotated on a regular basis. You should also configure your applications to use the latest version of the encrypted credentials when connecting to the RDS database. Which of the following is the MOST appropriate solution to secure the credentials?

- Store the credentials in AWS KMS.
- Store the credentials to AWS ACM.
- Store the credentials to Systems Manager Parameter Store with a `SecureString` data type.
- Use AWS Secrets Manager to store and encrypt the credentials and enable automatic rotation.

Incorrect

AWS Secrets Manager is an AWS service that makes it easier for you to manage secrets. *Secrets* can be database credentials, passwords, third-party API keys, and even arbitrary text. You can store and control access to these secrets centrally by using the Secrets Manager console, the Secrets Manager command line interface (CLI), or the Secrets Manager API and SDKs.

In the past, when you created a custom application that retrieves information from a database, you typically had to embed the credentials (the secret) for accessing the database directly in the application. When it came time to rotate the credentials, you had to do much more than just create new credentials. You had to invest time to update the application to use the new credentials. Then you had to distribute the updated application. If you had multiple applications that shared credentials and you missed updating one of them, the application would break. Because of this risk, many customers have chosen not to regularly rotate their credentials, which effectively substitutes one risk for another.



Secrets Manager enables you to replace hardcoded credentials in your code (including passwords), with an API call to Secrets Manager to retrieve the secret programmatically. This helps ensure that the secret can't be compromised by someone examining your code, because the secret simply isn't there. Also, you can configure Secrets Manager to automatically rotate the secret for you according to a schedule that you specify. This enables you to replace long-term secrets with short-term ones, which helps to significantly reduce the risk of compromise.

Hence, **using AWS Secrets Manager to store and encrypt the credentials and enabling automatic rotation** is the most appropriate solution for this scenario.

Storing the credentials to Systems Manager Parameter Store with a `SecureString` data type is incorrect because, by default, Systems Manager Parameter Store doesn't rotate its parameters which is one of the requirements in the above scenario.

Storing the credentials to AWS ACM is incorrect because it is just a managed private CA service that helps you easily and securely manage the lifecycle of your private certificates to allow SSL communication to your application. This is not a suitable service to store database or any other confidential credentials.

Storing the credentials in AWS KMS is incorrect because this only makes it easy for you to create and manage encryption keys and control the use of encryption across a wide range of AWS services. This is primarily used for encryption and not for hosting your credentials.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-paramstore.html>
<https://aws.amazon.com/blogs/compute/sharing-secrets-with-aws-lambda-using-aws-systems-manager-parameter-store/>

Check out these AWS Systems Manager and Secrets Manager Cheat Sheets:

<https://tutorialsdojo.com/aws-systems-manager/>
<https://tutorialsdojo.com/aws-secrets-manager/>

4. 4. Question

Category: CDA – Security

A startup recently launched a high-quality photo-sharing portal using Amazon Lightsail and Amazon S3. The team noticed that other external websites are linking and using the photos without permission. This situation has caused an increase in data transfer costs and potential revenue loss.

Which of the following is the MOST effective method to solve this issue?

- Enable cross-origin resource sharing (CORS) which allows cross-origin GET requests from all origins.
- Block the IP addresses of the offending websites using Network Access Control List.
- Configure the S3 bucket to remove public read access and use pre-signed URLs with expiry dates.
- Use an Amazon CloudFront web distribution with signed URLs or signed cookies.

Incorrect

A signed URL contains extra information, such as an expiration date and time, providing greater control over access to your content. This additional information is presented in a policy statement, which is derived from either a predefined (canned) policy or a personalized (custom) policy.

CloudFront signed cookies allow you to control who can access your content when you don't want to change your current URLs or when you want to provide access to multiple restricted files, such as all of the files in a website's subscribers' area. This topic explains the considerations when using signed cookies and describes how to set signed cookies using canned and custom policies.

The most effective method to control unauthorized access to the photos and manage data transfer costs is to use an Amazon CloudFront web distribution with signed URLs or signed cookies. This approach allows the startup to enforce access controls at the CDN layer, ensuring that only authorized users can access the content.

CloudFront X

Default cache behavior

Path pattern **Info**

Default (*)

Compress objects automatically **Info**

☐ No

☒ Yes

Viewer

Viewer protocol policy

☒ HTTP and HTTPS

☐ Redirect HTTP to HTTPS

☐ HTTPS only

Allowed HTTP methods

☒ GET, HEAD

☐ GET, HEAD, OPTIONS

☐ GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE

Restrict viewer access

If you restrict viewer access, viewers must use CloudFront signed URLs or signed cookies to access your content.

☐ No

☒ Yes

Trusted authorization type

☒ Trusted key groups (recommended)

☐ Trusted signer

Add key groups

Choose the key groups

Create key group

Additionally, CloudFront Functions can be used to validate referrer headers, adding another layer of protection by ensuring that only requests originating from the startup's own domain are allowed to access the content.

CloudFront X

CloudFront > Functions > Create function

Create function

Details

Name

Enter a name for the function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Description - optional

Enter a description for the function.

Runtime **Info**

Select the version of JavaScript for this function to run in.

☐ cloudfront-js-1.0

The legacy runtime version, which includes support for ECMAScript 5.1.

☒ cloudfront-js-2.0

This version supports JavaScript methods such as `async/await`. Note that if your function uses key value pairs, you must use this runtime version.

Cancel Create function

By using CloudFront with signed URLs or signed cookies, the startup can manage access effectively, providing a scalable solution that prevents unauthorized use while controlling data transfer costs. The use of CloudFront Functions to validate referrer headers adds an extra layer of security, ensuring that the photos are only accessible through authorized channels, thereby protecting the startup's content and reducing unnecessary data transfer costs.

Hence, the correct answer is: **Use an Amazon CloudFront web distribution with signed URLs or signed cookies.**

The option that says: **Enable cross-origin resource sharing (CORS) which allows cross-origin GET requests from all origins** is incorrect as this will typically make the problem worse since you are allowing any website or origin to fetch the objects in the S3 bucket. Although using CORS is a valid solution, it should be properly configured to only enable access to your trusted domains.

The option that says: **Configure the S3 bucket to remove public read access and use pre-signed URLs with expiry dates** is incorrect because it is not primarily scalable for large numbers of objects, as it would require generating pre-signed URLs for potentially thousands or millions of objects, making it impractical.

Blocking the IP addresses of the offending websites using Network Access Control List is incorrect because a quick change in IP address would easily bypass this configuration; hence, this is not an efficient method to implement.

References:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-signed-urls.html>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/SecurityAndPrivateContent.html>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html>

Check out this Amazon CloudFront Cheat Sheet:

<https://tutorialsdojo.com/amazon-cloudfront/>

5. Question

Category: CDA – Security

Your development team is currently developing a financial application in AWS. One of the requirements is to create and control the encryption keys used to encrypt your data using the envelope encryption strategy to comply with the strict IT security policy of the company.

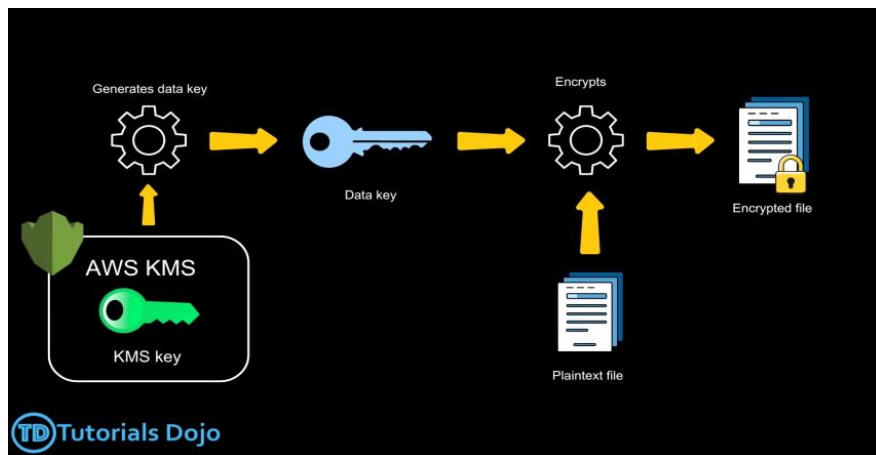
Which of the following correctly describes the process of envelope encryption?

- Encrypt plaintext data with a data key and then encrypt the data key with a top-level encrypted key.
- Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level plaintext data key.

- Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level encrypted data key.
- Encrypt plaintext data with a data key and then encrypt the data key with a top-level plaintext key.

Incorrect

When you encrypt your data, your data is protected, but you have to protect your encryption key. One strategy is to encrypt it. **Envelope encryption** is the practice of encrypting plaintext data with a data key and then encrypting the data key under another key.



You can even encrypt the data encryption key under another encryption key, and encrypt that encryption key under another encryption key. But, eventually, one key must remain in plaintext so you can decrypt the keys and your data. This top-level plaintext key encryption key is known as the *root key*.

AWS KMS helps you protect your encryption keys by storing and managing them securely. Root keys stored in AWS KMS, known as AWS KMS keys, never leave the AWS KMS FIPS validated hardware security modules unencrypted. To use an AWS KMS key, you must call AWS KMS.

Envelope encryption offers several benefits:

Protecting data keys

When you encrypt a data key, you don't have to worry about storing the encrypted data key, because the data key is inherently protected by encryption. You can safely store the encrypted data key alongside the encrypted data.

Encrypting the same data under multiple keys

Encryption operations can be time-consuming, particularly when the data being encrypted are large objects. Instead of re-encrypting raw data multiple times with different keys, you can re-encrypt only the data keys that protect the raw data.

Combining the strengths of multiple algorithms

In general, symmetric key algorithms are faster and produce smaller ciphertexts than public-key algorithms, but public-key algorithms provide inherent separation of roles and easier key management. Envelope encryption lets you combine the strengths of each strategy.

Therefore, the correct answer is: **Encrypt plaintext data with a data key and then encrypt the data key with a top-level plaintext key.**

The option that says: **Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level plaintext data key** is incorrect because you have to encrypt your plaintext data with a data key and not a KMS key. Moreover, the top-level plaintext key should be the root key and not the data key.

The option that says: **Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level encrypted data key** is incorrect because plaintext data should be encrypted with a data key, not a KMS key. Also, the top-level key should be plaintext, not encrypted.

The option that says: **Encrypt plaintext data with a data key and then encrypt the data key with a top-level encrypted key** is incorrect. While it is correct to encrypt plaintext data with a data key, the top-level key (root key) must be kept in plaintext and not be encrypted.

References:

<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#enveloping>

<https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>

Check out this AWS Key Management Service (KMS) Cheat Sheet:

<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

6. Question

Category: CDA – Security

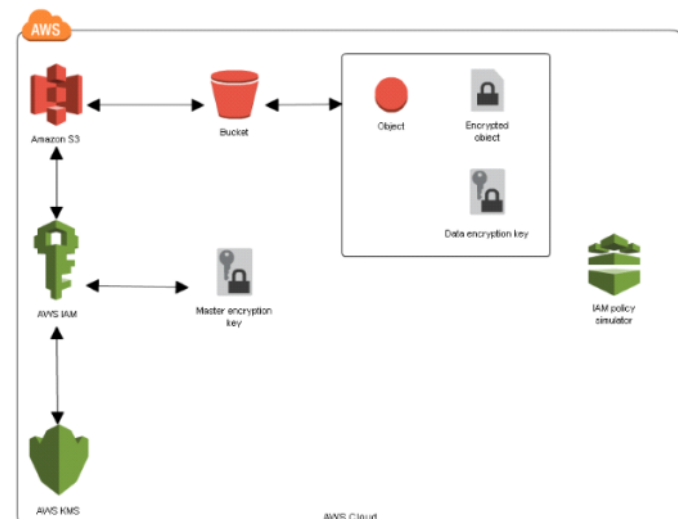
You are developing a new batch job for the enterprise application suite in your company, which is hosted in an Auto Scaling group of EC2 instances behind an ELB. The application is using an S3 bucket configured with Server-Side Encryption with AWS KMS Keys (SSE-KMS). The batch job must upload files to the bucket using the default AWS KMS key to protect the data at rest.

What should you do to satisfy this requirement with the LEAST amount of configuration?

- Include the `x-amz-server-side-encryption` header with a value of `AES256` in your upload request.
- Include the `x-amz-server-side-encryption` header with a value of `aws:kms` as well as the `x-amz-server-side-encryption-aws-kms-key-id` header containing the ID of the default AWS KMS key in your upload request.
- Include the `x-amz-server-side-encryption-customer-algorithm`, `x-amz-server-side-encryption-customer-key`, and `x-amz-server-side-encryption-customer-key-MD5` headers with appropriate values in the upload request.
- Include the `x-amz-server-side-encryption` header with a value of `aws:kms` in your upload request.

Incorrect

Server-side encryption is about protecting data at rest. AWS Key Management Service (AWS KMS) is a service that combines secure, highly available hardware and software to provide a key management system scaled for the cloud. AWS KMS uses KMS keys to encrypt your Amazon S3 objects. You use AWS KMS via the [AWS Management Console](#) or [AWS KMS APIs](#) to centrally create encryption keys, define the policies that control how keys can be used, and audit key usage to prove they are being used correctly. You can use these keys to protect your data in Amazon S3 buckets.



The first time you add an SSE-KMS-encrypted object to a bucket in a region, a default KMS key is created for you automatically. This key is used for SSE-KMS encryption unless you select a KMS key that you created separately using AWS Key Management Service. Creating your own KMS key gives you more flexibility, including the ability to create, rotate, disable, and define access controls and audit the encryption keys used to protect your data.

To upload an object to the S3 bucket, which uses SSE-KMS, you have to send a request with an `x-amz-server-side-encryption` header with the value of `aws:kms`. There's also an optional `x-amz-server-side-encryption-aws-kms-key-id` header, which specifies the ID of the AWS KMS master encryption key that was used for the object. The Amazon S3 API also supports encryption context with the `x-amz-server-side-encryption-context` header.

When you upload an object, you can specify the KMS key using the `x-amz-server-side-encryption-aws-kms-key-id` header. If the header is not present in the request, Amazon S3 assumes the default KMS key. Regardless, the KMS key ID that Amazon S3 uses for object encryption must match the KMS key ID in the policy, otherwise, Amazon S3 denies the request.

Hence, the correct is: **Include the `x-amz-server-side-encryption` header with a value of `aws:kms` in your upload request.**

The option that says: **Including the `x-amz-server-side-encryption-customer-algorithm`, `x-amz-server-side-encryption-customer-key` and `x-amz-server-side-encryption-customer-key-MD5` headers with appropriate values in the upload request** is incorrect because these headers are only required if your S3 bucket is using Server-Side Encryption with Customer-Provided Keys (SSE-C).

The option that says: **Including the `x-amz-server-side-encryption` header with a value of `aws:kms` as well as the `x-amz-server-side-encryption-aws-kms-key-id` header containing the ID of the default AWS KMS key in your upload request** is incorrect. Although this is a valid option, you actually don't need to add the `x-amz-server-side-encryption-aws-kms-key-id` header if you will be using the default AWS KMS key. Take note that the scenario explicitly mentioned to provide a solution with the LEAST amount of configuration.

The option that says: **Including the `x-amz-server-side-encryption` header with a value of `AES256` in your upload request** is incorrect because the value should be set as `aws:kms` instead. The value of `AES256` is only applicable for SSE-S3 and SSE-C.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingKMSEncryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/API/RESTObjectPUT.html#RESTObjectPUT-responses-examples>
<https://aws.amazon.com/blogs/security/how-to-prevent-uploads-of-unencrypted-objects-to-amazon-s3/>

Check out these Amazon S3 and AWS KMS Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>
<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

7. 7. Question

Category: CDA – Security

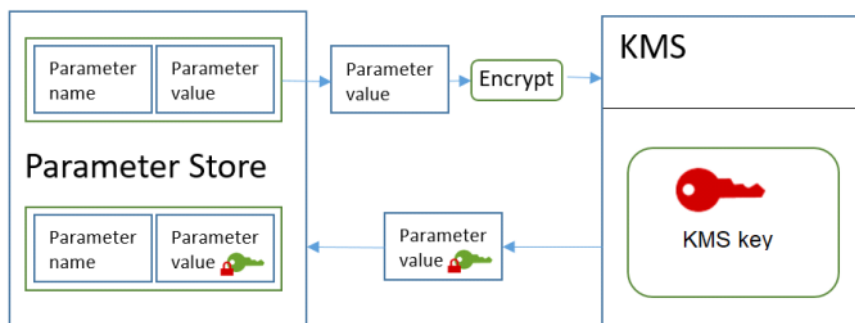
Your team is developing a serverless application, which is composed of multiple Lambda functions which process data from an SQS queue and stores the results to an RDS database. To comply with the strict IT policy of the company, you were instructed to configure these functions to share the same connection string that should be properly secured and encrypted.

What should you do to protect, encrypt, and share your database credentials in AWS?

- Use AWS Systems Manager Parameter Store as a Secure String Parameter.
- Encrypt the database credentials and store them in an S3 bucket which the Lambda functions can fetch.
- Store the database credentials as environment variables with KMS encryption which will be shared by the Lambda functions.
- Use IAM DB Authentication in RDS to allow encrypted connections from each Lambda function.

Incorrect

AWS Systems Manager Parameter Store provides secure, hierarchical storage for configuration data management and secrets management. You can store data such as passwords, database strings, and license codes as parameter values. You can store values as plain text or encrypted data. You can then reference values by using the unique name that you specified when you created the parameter.



Parameter Store offers the following benefits and features:

- Use a secure, scalable, hosted secrets management service (No servers to manage).
- Improve your security posture by separating your data from your code.
- Store configuration data and secure strings in hierarchies and track versions.
- Control and audit access at granular levels.
- Configure change notifications and trigger automated actions.
- Tag parameters individually, and then secure access from different levels, including operational, parameter, Amazon EC2 tag, or path levels.
- Reference AWS Secrets Manager secrets by using Parameter Store parameters.

Hence, the correct solution for this scenario is to **use AWS Systems Manager Parameter Store as a Secure String Parameter.**

Encrypting the database credentials and storing them in an S3 bucket which the Lambda functions can fetch is incorrect because it is a security risk to store sensitive database passwords and credentials in S3, even though the data is encrypted. A more suitable and secure way is to use the AWS Secrets Manager or the Systems Manager Parameter Store.

Storing the database credentials as environment variables with KMS encryption which will be shared by the Lambda functions is incorrect because even though the credentials will be encrypted, these environment variables will only be used by an individual Lambda function and cannot be shared.

Using IAM DB Authentication in RDS to allow encrypted connections from each Lambda function is incorrect. While using IAM DB Authentication can significantly enhance security for database connections in AWS environments, it's more about controlling access rather than managing the sharing of encrypted credentials like a database connection string across multiple applications or services. Additionally, this method is limited to certain types of databases supported by AWS, such as Amazon RDS for MySQL and PostgreSQL, and might not be available for all database engines. Lastly, in the scenario where Lambda functions need to share and securely access a connection string, other AWS services like the Systems Manager Parameter Store would be more appropriate to meet all the specified needs.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-paramstore.html>
<https://aws.amazon.com/blogs/compute/sharing-secrets-with-aws-lambda-using-aws-systems-manager-parameter-store/>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-systems-manager/>

8. 8. Question

Category: CDA – Security

A software engineer is building a serverless application in AWS consisting of Lambda, API Gateway, and DynamoDB. She needs to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML to determine the caller's identity.

Which of the features of API Gateway is the MOST suitable one that she should use to build this feature?

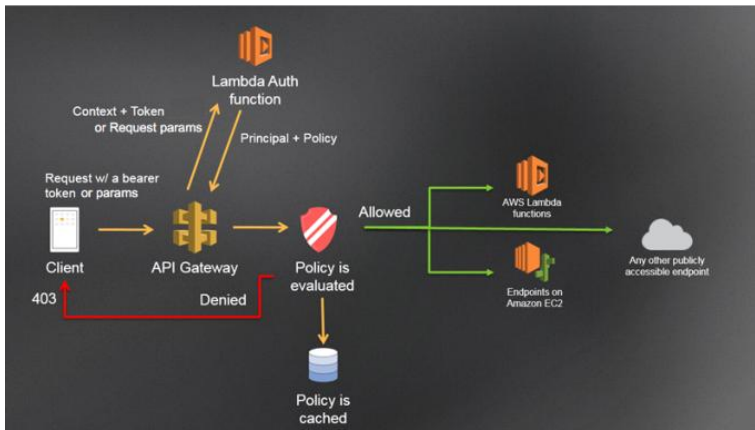
- Resource Policy
- Lambda Authorizers
- Cross-Origin Resource Sharing (CORS)
- Cross-Account Lambda Authorizer

Incorrect

A Lambda authorizer is an API Gateway feature that uses a Lambda function to control access to your API. When a client makes a request to one of your API's methods, API Gateway calls your Lambda authorizer, which takes the caller's identity as input and returns an IAM policy as output.

There are two types of Lambda authorizers:

- A **token-based** Lambda authorizer (also called a **TOKEN** authorizer) receives the caller's identity in a bearer token, such as a JSON Web Token (JWT) or an OAuth token.
- A **request parameter-based** Lambda authorizer (also called a **REQUEST** authorizer) receives the caller's identity in a combination of headers, query string parameters, stageVariables, and \$context variables.



It is possible to use an AWS Lambda function from an AWS account that is different from the one in which you created your Lambda authorizer function by using a Cross-Account Lambda Authorizer. Therefore, the correct answer in this scenario is to use **Lambda Authorizers** since this feature is useful if you want to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML.

Resource Policy is incorrect because this is simply a JSON policy document that you attach to an API to control whether a specified principal (typically an IAM user or role) can invoke the API. This can't be used to implement a custom authorization scheme.

Cross-Origin Resource Sharing (CORS) is incorrect because this just defines a way for client web applications that are loaded in one domain to interact with resources in a different domain.

Cross-Account Lambda Authorizer is incorrect because this just enables you to use an AWS Lambda function from a different AWS account as your API authorizer function. Moreover, this is not a valid Lambda authorizer type.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-lambda-authorizer-input.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-lambda-authorizer-cross-account-lambda-authorizer.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

9. Question

Category: CDA – Security

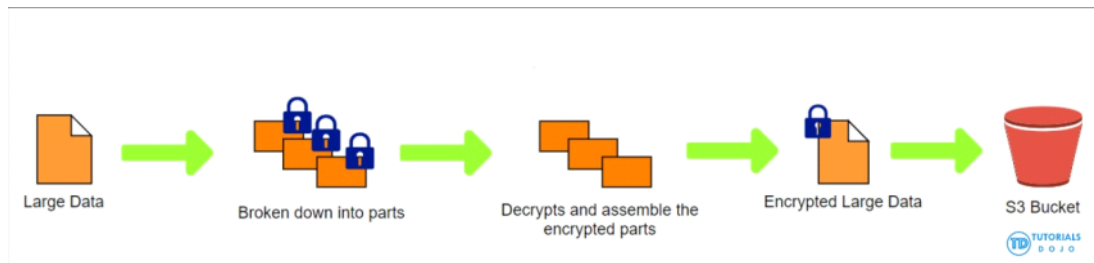
A developer runs a shell script that uses the `aws s3 cp` CLI to upload a large file to an S3 bucket. The S3 bucket is configured with Server-side encryption with AWS Key Management Service (SSE-KMS). An `Access Denied` error always shows up whenever the developer uploads a file with a size of 100 GB or more. However, whenever he uploads a smaller file, the request succeeds.

Which of the following are possible reasons why this issue is happening? (Select TWO.)

- The developer does not have the `kms:Decrypt` permission.
- The developer's IAM permission has an attached inline policy that restricts him from uploading a file to S3 with a size of 100 GB or more.
- The AWS CLI S3 commands perform a multipart upload when the file is large.
- The maximum size that can be encrypted in KMS is only 100 GB.
- The developer does not have the `kms:Encrypt` permission.

Incorrect

If you are getting an `Access Denied` error when trying to upload a large file to your S3 bucket with an upload request that includes an AWS KMS key, then you have to confirm that you have permission to perform `kms:Decrypt` actions on the AWS KMS key that you're using to encrypt the object.



To perform a multipart upload with encryption using an AWS KMS key, the requester must have permission to the `kms:Decrypt` and `kms:GenerateDataKey*` actions on the key. These permissions are required because Amazon S3 must decrypt and read data from the encrypted file parts before it completes the multipart upload.

Hence, the correct answers in this scenario are:

- The AWS CLI S3 commands perform a multipart upload when the file is large.
- The developer does not have the `kms:Decrypt` permission.

The option that says: **The developer does not have the `kms:Encrypt` permission** is incorrect. This permission is not necessary for you to have as it's typically used for direct encryption requests with the KMS key and is only applicable for small data up to 4KB. Amazon S3 operates differently; it generates a data key from your KMS key to encrypt files. Behind the scenes, Amazon S3 actually generates a data key from your KMS key to encrypt files.

The option that says: **The developer's IAM permission has an attached inline policy that restricts him from uploading a file to S3 with a size of 100 GB or more** is incorrect because inline policies are just policies that you create, manage, and embed directly into a single user, group, or role. This is unlikely to happen as there is no direct way to restrict a user from uploading a file with a specific size constraint.

The option that says: **The maximum size that can be encrypted in KMS is only 100 GB** is incorrect because there is no such limitation in KMS.

References:

<https://aws.amazon.com/premiumsupport/knowledge-center/s3-large-file-encryption-kms-key/>
<https://docs.aws.amazon.com/cli/latest/userguide/cli-services-s3-commands.html#using-s3-commands-managing-objects>

Check out this AWS Key Management Service (KMS) Cheat Sheet:

<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

10. Question

Category: CDA – Security

A programmer is developing a Node.js application that will be run on a Linux server in their on-premises data center. The application will access various AWS services such as S3, DynamoDB, and ElastiCache using the AWS SDK. Which of the following is the MOST suitable way to provide access for the developer to accomplish the specified task?

- Create an IAM role with the appropriate permissions to access the required AWS services and assign the role to the on-premises Linux server. Whenever the application needs to access any AWS services, request for temporary security credentials from STS using the `AssumeRole` API.
- Go to the AWS Console and create a new IAM user with programmatic access. In the application server, create the credentials file at `~/.aws/credentials` with the access keys of the IAM user.
- Create an IAM role with the appropriate permissions to access the required AWS services. Assign the role to the on-premises Linux server.
- Go to the AWS Console and create a new IAM User with the appropriate permissions. In the application server, create the credentials file at `~/.aws/credentials` with the username and the hashed password of the IAM User.

Incorrect

If you have resources that are running inside AWS that need programmatic access to various AWS services, then the best practice is always to use IAM roles. However, applications running outside of an AWS environment

will need access keys for programmatic access to AWS resources. For example, monitoring tools running on-premises and third-party automation tools will need access keys. Access keys are long-term credentials for an IAM user or the AWS account root user. You can use access keys to sign programmatic requests to the AWS CLI or AWS API (directly or using the AWS SDK).

Set user details

You can add multiple users at once with the same access type and permissions. [Learn more](#)

User name*

[Add another user](#)

Select AWS access type

Select how these users will access AWS. Access keys and autogenerated passwords are provided in the last step. [Learn more](#)

Access type* ☒ **Programmatic access**
Enables an **access key ID** and **secret access key** for the AWS API, CLI, SDK, and other development tools.

☐ **AWS Management Console access**
Enables a **password** that allows users to sign-in to the AWS Management Console.

In order to use the AWS SDK for your application, you have to create your credentials file first at `~/.aws/credentials` for Linux servers or at `C:\Users\USER_NAME\.aws\credentials` for Windows users and then save your access keys.

Hence, the correct answer is: **Go to the AWS Console and create a new IAM user with programmatic access. In the application server, create the credentials file at `~/.aws/credentials` with the access keys of the IAM user.**

The option that says: **Create an IAM role with the appropriate permissions to access the required AWS services and assign the role to the on-premises Linux server. Whenever the application needs to access any AWS services, request for temporary security credentials from STS using the AssumeRole API** is incorrect because the scenario says that the application is running in a Linux server on-premises and not on an EC2 instance. You cannot directly assign an IAM Role to a server on your on-premises data center. Although it may be possible to use a combination of STS and IAM Role, the use of access keys for AWS SDK is still preferred, especially if the application server is on-premises.

The option that says: **Create an IAM role with the appropriate permissions to access the required AWS services. Assign the role to the on-premises Linux server** is also incorrect because, just as mentioned above, the use of an IAM Role is not a suitable solution for this scenario.

The option that says: **Go to the AWS Console and create a new IAM User with the appropriate permissions. In the application server, create the credentials file at `~/.aws/credentials` with the username and the hashed password of the IAM User** is incorrect. An IAM user's username and password can only be used to interact with AWS via its Management Console. These credentials are intended for human use and are not suitable for use in automated systems, such as applications and scripts that make programmatic calls to AWS services.

References:

<https://aws.amazon.com/developers/getting-started/nodejs/>
<https://docs.aws.amazon.com/general/latest/gr/aws-sec-cred-types.html#access-keys-and-secret-access-keys>
<https://aws.amazon.com/blogs/security/guidelines-for-protecting-your-aws-account-while-using-programmatic-access/>

Check out this AWS IAM Cheat Sheet:

<https://tutorialsdemo.com/aws-identity-and-access-management-iam/>

11. Question

Category: CDA – Security

A company uses AWS Systems Manager (SSM) Parameter Store to manage configuration details for multiple applications. The parameters are currently stored in the Standard tier. The company wants its operations team to be notified if there are sensitive parameters that haven't been rotated within 90 days.

Which must be done to meet the requirement?

- Convert the sensitive parameters from Standard tier into Advanced tier. Set a `ExpirationNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.
- Convert the sensitive parameters from Standard tier into Advanced tier. Set a `NoChangeNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.
- Configure a `NoChangeNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.
- Set up an Amazon EventBridge (Amazon CloudWatch Events) event pattern that captures SSM Parameter-related events. Use Amazon SNS to send notifications.

Incorrect

Parameter policies help you manage a growing set of parameters by allowing you to assign specific criteria to a parameter, such as an expiration date or time to live. Parameter policies are especially helpful in forcing you to update or delete passwords and configuration data stored in Parameter Store, a capability of AWS Systems Manager. Take note that parameter policies are only available for parameters in the **Advanced tier**.

Parameter Store offers the following types of policies:

Expiration – deletes the parameter at a specific date

ExpirationNotification – sends an event to Amazon EventBridge (Amazon CloudWatch Events) when the specified expiration time is reached.

NoChangeNotification – sends an event to Amazon EventBridge (Amazon CloudWatch Events) when a parameter has not been modified for a specified period of time.

Parameter policies — Optional

Parameter expiration
Specify how long the parameter is available in Parameter Store, as either a point-in-time or lifetime. The parameter is deleted after the specified date.

☒ No expiration policy
☐ Absolute time
☐ Relative time

Notification policies
Create notification policies to send notifications from CloudWatch Events. Notifications can be based on when this parameter expires, or if the parameter has not changed for a given period of time.

Policy type	Policy value	Value type	
Notify if no change h...	90	Days	Remove policy

[Add policy](#)

[Cancel](#) [Save changes](#)

The **NoChangeNotification** policy sends a notification based on the `LastModifiedTime` attribute of the parameter. If you change or edit a parameter, the system resets the notification time period based on the new value of `LastModifiedTime`. In the scenario's case, we want to be notified if specific parameters were not rotated in the last 90 days.

In the scenario, the goal is to be notified if specific sensitive parameters have not been rotated within the past 90 days. Configuring the **NoChangeNotification** policy with a value of 90 days allows SSM to emit a notification to EventBridge whenever the `LastModifiedTime` of the sensitive parameters exceeds the specified time frame. However, setting the notification policy alone is not enough. You must configure Amazon EventBridge (Amazon CloudWatch Events) to capture the emitted events and route them to an Amazon SNS topic.

Hence, the correct answer is: **Convert the sensitive parameters from Standard tier into Advanced tier. Set a `NoChangeNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.**

The option that says: **Configure a `NoChangeNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS** is incorrect because notification policies are not supported in the Standard tier. You must convert the parameters first into the Advanced tier.

The option that says: **Convert the sensitive parameters from Standard tier into Advanced tier. Set a `ExpirationNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS** is incorrect because the `ExpirationNotification` policy is for notifying when a parameter is about to expire, not when it hasn't been rotated. In this case, the `NoChangeNotification` policy should be used instead.

The option that says: **Set up an Amazon EventBridge (Amazon CloudWatch Events) event pattern that captures SSM Parameter-related events. Use Amazon SNS to send notifications** is incorrect. A notification policy must be enabled as well, otherwise, Amazon EventBridge (Amazon CloudWatch Events) won't be able to receive any notifications.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-policies.html>
<https://aws.amazon.com/about-aws/whats-new/2019/04/aws-systems-manager-parameter-store-introduces-advanced-parameters/>

Check out this cheat sheet on AWS Secrets Manager vs Systems Manager Parameter Store:

<https://tutorialsdemo.com/aws-secrets-manager-vs-systems-manager-parameter-store/>

12. Question

Category: CDA – Security

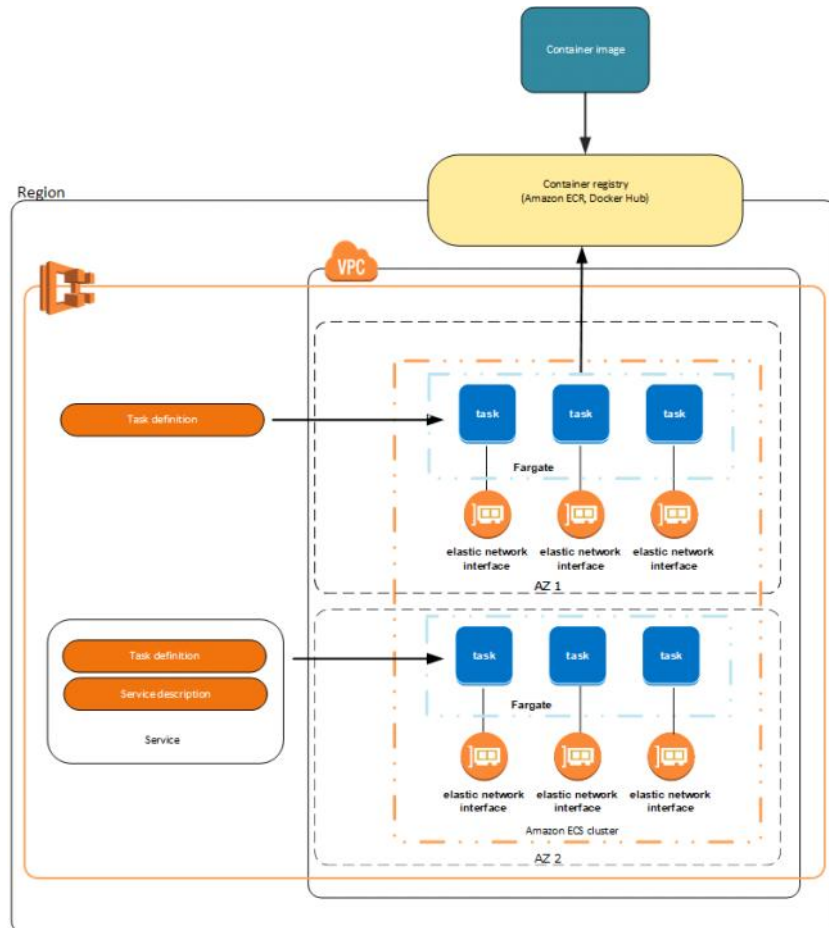
A developer is building an application that will be hosted in ECS and must be configured to run tasks and services using the Fargate launch type. The application will have four different tasks, each of which will access different AWS resources than the others.

Which of the following is the MOST efficient solution that can provide your application in ECS access to the required AWS resources?

- Create an IAM Group with all the required permissions and attach them to each of the 4 ECS tasks.
- Create 4 different Service-Linked Roles with the required permissions and attach them to each of the 4 ECS tasks.
- Create 4 different Container Instance IAM Roles with the required permissions and attach them to each of the 4 ECS tasks.
- Create 4 different IAM Roles with the required permissions and attach them to each of the 4 ECS tasks.

Incorrect

By default, IAM users don't have permission to create or modify Amazon ECS resources or perform tasks using the Amazon ECS API. This means that they also can't do so using the Amazon ECS console or the AWS CLI. To allow IAM users to create or modify resources and perform tasks, you must create IAM policies. Policies grant IAM users permission to use specific resources and API actions. Then, attach those policies to the IAM users or groups that require those permissions.



When you attach a policy to a user or group of users, it allows or denies the users permission to perform the specified tasks on the specified resources. Likewise, Amazon ECS container instances make calls to the Amazon ECS and Amazon EC2 APIs on your behalf, so they need to authenticate with your credentials. This authentication is accomplished by creating an IAM role for your container instances and associating that role with your container instances when you launch them.

If you use an Elastic Load Balancing load balancer with your Amazon ECS services, calls to the Amazon EC2 and Elastic Load Balancing APIs are made on your behalf to register and deregister container instances with your load balancers.

Hence, the most suitable solution in this scenario is: **Create 4 different IAM Roles with the required permissions and attach them to each of the 4 ECS tasks.**

The option that says: **Creating an IAM Group with all the required permissions and attaching them to each of the 4 ECS tasks** is incorrect because you cannot directly attach an IAM Group to an ECS Task. Attaching an IAM Role is a more suitable solution in this scenario and not an IAM Group.

The option that says: **Creating 4 different Container Instance IAM Roles with the required permissions and attaching them to each of the 4 ECS tasks** is incorrect because a Container Instance IAM Role only applies if you are using the EC2 launch type. Take note that the scenario says that the application will be using a Fargate launch type.

The option that says: **Creating 4 different Service-Linked Roles with the required permissions and attaching them to each of the 4 ECS tasks** is incorrect because a service-linked role is a unique type of IAM role that is linked directly to Amazon ECS itself, not on the ECS task. Service-linked roles are predefined by Amazon ECS and include all the permissions that the service requires to call other AWS services on your behalf.

References:

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task_IAM_role.html
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-iam-roles.html#create_task_iam_policy_and_role
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/instance_IAM_role.html

Check out this Amazon ECS Cheat Sheet:

<https://tutorialsdojo.com/amazon-elastic-container-service-amazon-ecs/>

13. Question

Category: CDA – Security

A financial mobile application has a serverless backend API which consists of DynamoDB, Lambda, and Cognito. Due to the confidential financial transactions handled by the mobile application, there is a new requirement provided by the company to add a second authentication method that doesn't rely solely on user name and password.

Which of the following is the MOST suitable solution that the developer should implement?

- Integrate multi-factor authentication (MFA) to a user pool in Cognito to protect the identity of your users.
- Create a custom application that integrates with Amazon Cognito which implements the second layer of authentication.
- Use a new IAM policy to a user pool in Cognito.
- Use Cognito with SNS to allow additional authentication via SMS.

Incorrect

You can add multi-factor authentication (MFA) to a user pool to protect the identity of your users. MFA adds a second authentication method that doesn't rely solely on usernames and passwords. You can choose to use SMS text messages, or time-based one-time (TOTP) passwords as second factors in signing in your users. You can also use adaptive authentication with its risk-based model to predict when you might need another authentication factor. It's part of the user pool's advanced security features, which also include protections against compromised credentials.

Edit multi-factor authentication (MFA) Info

Amazon Cognito provides your app users with additional authentication factors using SMS messages and time-based one-time passwords (TOTP).

Multi-factor authentication
Configure secure access to your app by enforcing multi-factor authentication (MFA) during the user sign-in process. MFA settings are applied to all app clients.

MFA enforcement Info

☐ Require MFA - Recommended
Users must provide an additional authentication factor when signing in.

☒ Optional MFA
Users can sign in with a single authentication factor, and can choose to add additional authentication factors.

☐ No MFA
Users can only sign in with a single authentication factor. This is the least secure option.

MFA methods Info
Choose the MFA methods that are allowed in your user pool. TOTP-based MFA offers a higher level of security. Recipient message and data rates apply.

☒ Authenticator apps
Users can authenticate with a TOTP from an authenticator app such as Authy or Google Authenticator.

☒ SMS message
Users can authenticate with a code sent by SMS message to a verified phone number. SMS messages are charged separately by Amazon SNS. [Learn more about pricing](#) This option must be selected because SMS is configured.

Cancel Save changes

Multi-factor authentication (MFA) increases security for your app by adding another authentication method, and not relying solely on user name and password. You can choose to use SMS text messages, or time-based one-time (TOTP) passwords as second factors in signing in your users.

With adaptive authentication, you can configure your user pool to require second-factor authentication in response to an increased risk level.

Hence, the correct answer in this scenario is to **integrate multi-factor authentication (MFA) to a user pool in Cognito to protect the identity of your users**.

Creating a custom application that integrates with Amazon Cognito which implements the second layer of authentication is incorrect. Although this option is viable, it is not the most suitable solution in this scenario since you can simply use MFA as a second-factor authentication for the mobile app.

Using a new IAM policy to a user pool in Cognito is incorrect because an IAM Policy alone cannot implement a second-factor authentication. You have to configure Cognito to use MFA instead.

Using Cognito with SNS to allow additional authentication via SMS is incorrect. Although this is part of the MFA setup, using this solution alone is not enough if you didn't enable MFA in the first place.

References:

<https://docs.aws.amazon.com/cognito/latest/developerguide/managing-security.html>
<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools.html>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

14. Question

Category: CDA – Security

A developer is building the cloud architecture of an application which will be hosted in a large EC2 instance. The application will process the data and it will upload results to an S3 bucket.

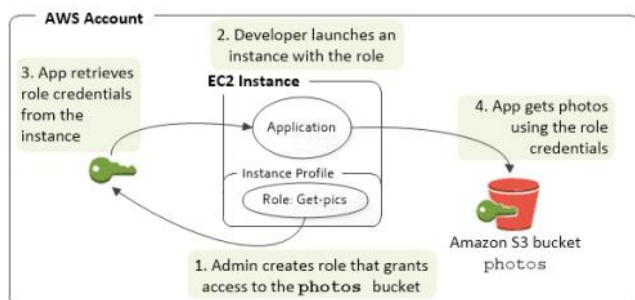
Which of the following is the SAFEST way to implement this architecture?

- Use an IAM Role to grant the application the necessary permissions to upload data to S3.
- Use an IAM Inline Policy to grant the application the necessary permissions to upload data to S3.
- Install the AWS CLI then use it to upload the results to S3.
- Store the access keys in the instance then use the AWS SDK to upload the results to S3.

Incorrect

Applications that run on an EC2 instance must include AWS credentials in their AWS API requests. You could have your developers store AWS credentials directly within the EC2 instance and allow applications in that instance to use those credentials. But developers would then have to manage the credentials and ensure that they securely pass the credentials to each instance and update each EC2 instance when it's time to rotate the credentials. That's a lot of additional work.

Instead, you can and should use an IAM role to manage *temporary* credentials for applications that run on an EC2 instance. When you use a role, you don't have to distribute long-term credentials (such as a username and password or access keys) to an EC2 instance. Instead, the role supplies temporary permissions that applications can use when they make calls to other AWS resources. When you launch an EC2 instance, you specify an IAM role to associate with the instance. Applications that run on the instance can then use the role-supplied temporary credentials to sign API requests.



Using roles to grant permissions to applications that run on EC2 instances requires a bit of extra configuration. An application running on an EC2 instance is abstracted from AWS by the virtualized operating system. Because of this extra separation, an additional step is needed to assign an AWS role and its associated permissions to an EC2 instance and make them available to its applications. This extra step is the creation of an *instance profile* that is attached to the instance. The instance profile contains the role and can provide the role's temporary credentials to an application that runs on the instance. Those temporary credentials can then be used in the application's API calls to access resources and to limit access to only those resources that the role specifies. Note that only one role can be assigned to an EC2 instance at a time, and all applications on the instance share the same role and permissions.

Using roles in this way has several benefits. Because role credentials are temporary and rotated automatically, you don't have to manage credentials, and you don't have to worry about long-term security risks. In addition, if you use a single role for multiple instances, you can make a change to that one role and the change is propagated automatically to all the instances.

Hence, **using an IAM Role to grant the application the necessary permissions to upload data to S3** is the correct answer for this scenario as this provides the safest way to integrate your application hosted in EC2 and S3.

Storing the access keys in the instance and then using the AWS SDK to upload the results to S3 is incorrect because this will expose the AWS access credentials to all users who have access to the EC2 instance. Since this option entails a security risk, this is incorrect as is not the safest method.

Installing the AWS CLI then using it to upload the results to S3 is incorrect. Although this option is valid, this method also presents a security risk just as shown above. By default, an AWS CLI requires you to store the AWS access keys in your instance which will be used in executing the commands. Hence, this option is incorrect.

Using an IAM Inline Policy to grant the application the necessary permissions to upload data to S3 is incorrect because inline policies are useful if you want to maintain a strict one-to-one relationship between a policy and the principal entity that it's applied to. This option doesn't provide a secure way of allowing the application that is hosted in EC2 to upload data to an S3 bucket. You should use an IAM Role instead.

References:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_switch-role-ec2.html
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html>

Check out this AWS Identity & Access Management (IAM) Cheat Sheet:

<https://tutorialsdojo.com/aws-identity-and-access-management-iam/>

15. Question

Category: CDA – Security

To improve their information security management system (ISMS), a company recently released a new policy which requires all database credentials to be encrypted and be automatically rotated to avoid unauthorized access.

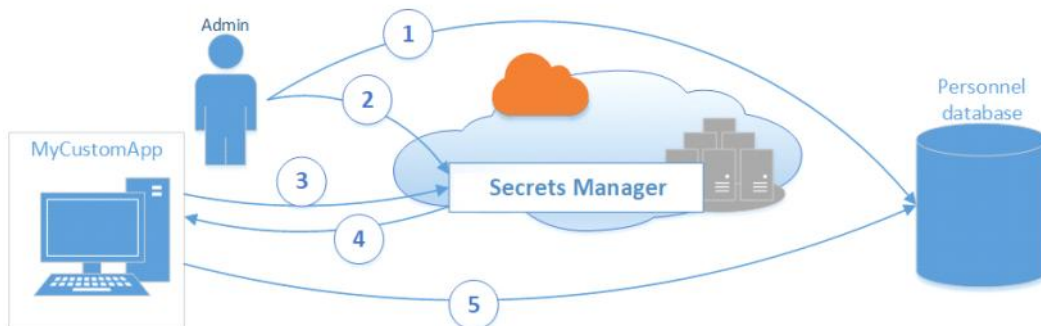
Which of the following is the MOST appropriate solution to secure the credentials?

- Enable IAM DB authentication which rotates the credentials by default.
- Create a secret in AWS Secrets Manager and enable automatic rotation of the database credentials.
- Create a parameter to the Systems Manager Parameter Store using the PutParameter API with a type of SecureString.
- Create an IAM Role which has full access to the database. Attach the role to the services which require access.

Incorrect

AWS Secrets Manager is an AWS service that makes it easier for you to manage secrets. *Secrets* can be database credentials, passwords, third-party API keys, and even arbitrary text. You can store and control access to these secrets centrally by using the Secrets Manager console, the Secrets Manager command line interface (CLI), or the Secrets Manager API and SDKs.

In the past, when you created a custom application that retrieves information from a database, you typically had to embed the credentials (the secret) for accessing the database directly in the application. When it came time to rotate the credentials, you had to do much more than just create new credentials. You had to invest time to update the application to use the new credentials. Then you had to distribute the updated application. If you had multiple applications that shared credentials and you missed updating one of them, the application would break. Because of this risk, many customers have chosen not to regularly rotate their credentials, which effectively substitutes one risk for another.



Secrets Manager enables you to replace hardcoded credentials in your code (including passwords), with an API call to Secrets Manager to retrieve the secret programmatically. This helps ensure that the secret can't be compromised by someone examining your code, because the secret simply isn't there. Also, you can configure Secrets Manager to automatically rotate the secret for you according to a schedule that you specify. This enables you to replace long-term secrets with short-term ones, which helps to significantly reduce the risk of compromise.

Hence, **creating a secret in AWS Secrets Manager and enabling automatic rotation of the database credentials** is the most appropriate solution for this scenario.

The option that says: **Create a parameter to the Systems Manager Parameter Store using the PutParameter API with a type of SecureString** is incorrect because, by default, Systems Manager Parameter Store doesn't rotate its parameters.

The option that says: **Enable IAM DB authentication which rotates the credentials by default** is incorrect because this solution only enables the service to connect to Amazon RDS with IAM credentials. It doesn't have the capability to rotate the credentials like what AWS Secrets Manager does to its secrets.

The option that says: **Create an IAM Role which has full access to the database. Attach the role to the services which requires access** is incorrect because although IAM Role is a preferred way to grant access to certain services, this solution doesn't rotate the keys/credentials.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-paramstore.html>
<https://aws.amazon.com/blogs/compute/sharing-secrets-with-aws-lambda-using-aws-systems-manager-parameter-store/>

Check out this AWS Secrets Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-secrets-manager/>

16. Question

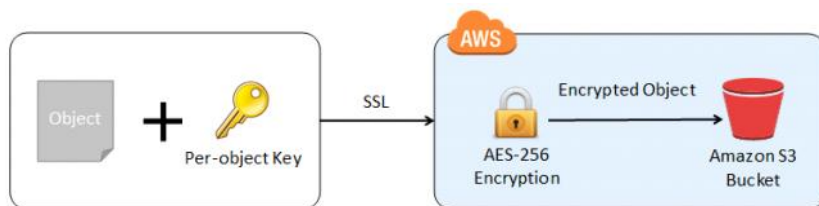
Category: CDA – Security

32x-amz-server-side-encryption and x-amz-server-side-encryption-aws-kms-key-id headers

- x-amz-server-side-encryption, x-amz-server-side-encryption-customer-key and x-amz-server-side-encryption-customer-key-MD5 headers
- x-amz-server-side-encryption-customer-key header only

Incorrect

Server-side encryption is about protecting data at rest. Using server-side encryption with customer-provided encryption keys (SSE-C) allows you to set your own encryption keys. With the encryption key you provide as part of your request, Amazon S3 manages both the encryption, as it writes to disks, and decryption, when you access your objects. Therefore, you don't need to maintain any code to perform data encryption and decryption. The only thing you do is manage the encryption keys you provide.



When you upload an object, Amazon S3 uses the encryption key you provide to apply AES-256 encryption to your data and removes the encryption key from memory. It is important to note that Amazon S3 does not store the encryption key you provide. Instead, it is stored in a randomly salted HMAC value of the encryption key in order to validate future requests. The salted HMAC value cannot be used to derive the value of the encryption key or to decrypt the contents of the encrypted object. That means, if you lose the encryption key, you lose the object.

When you retrieve an object, you must provide the same encryption key as part of your request. Amazon S3 first verifies that the encryption key you provided matches, and then decrypts the object before returning the object data to you.

When using server-side encryption with customer-provided encryption keys (SSE-C), you must provide encryption key information using the following request headers:

x-amz-server-side-encryption-customer-algorithm – This header specifies the encryption algorithm. The header value must be "AES256".

x-amz-server-side-encryption-customer-key – This header provides the 256-bit, base64-encoded encryption key for Amazon S3 to use to encrypt or decrypt your data.

x-amz-server-side-encryption-customer-key-MD5 – This header provides the base64-encoded 128-bit MD5 digest of the encryption key according to RFC 1321. Amazon S3 uses this header for a message integrity check to ensure the encryption key was transmitted without error.

Hence, the correct answer is: **x-amz-server-side-encryption-customer-algorithm, x-amz-server-side-encryption-customer-key and x-amz-server-side-encryption-customer-key-MD5 headers.**

The option that says: **Including the x-amz-server-side-encryption and x-amz-server-side-encryption-aws-kms-key-id headers in the upload request** is incorrect because these headers are primarily used in Server-Side Encryption with AWS KMS Keys (SSE-KMS) and not for Server-Side Encryption with Customer-Provided Keys (SSE-C).

The option that says: **Including the x-amz-server-side-encryption, x-amz-server-side-encryption-customer-key and x-amz-server-side-encryption-customer-key-MD5 headers** is incorrect because the x-amz-server-side-encryption header is not used in SSE-C encryption. This should be replaced with the x-amz-server-side-encryption-customer-algorithm header.

The option that says: **Including just the x-amz-server-side-encryption-customer-key header only** is incorrect because you have to include the x-amz-server-side-encryption-customer-algorithm and x-amz-server-side-encryption-customer-key-MD5 headers as well to upload the objects to the S3 bucket with SSE-C encryption.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingKMSEncryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/API/RESTObjectPUT.html#RESTObjectPUT-responses-examples>
<https://aws.amazon.com/blogs/security/how-to-prevent-uploads-of-unencrypted-objects-to-amazon-s3/>

Check out these Amazon S3 and AWS KMS Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>
<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

17. Question

Category: CDA – Security

A company is currently in the process of integrating their on-premises data center to their cloud infrastructure in AWS. One of the requirements is to integrate the on-premises Lightweight Directory Access Protocol (LDAP)

directory service to their AWS VPC using IAM.

Which of the following provides the MOST suitable solution to implement if the identity store that they are using is not compatible with SAML?

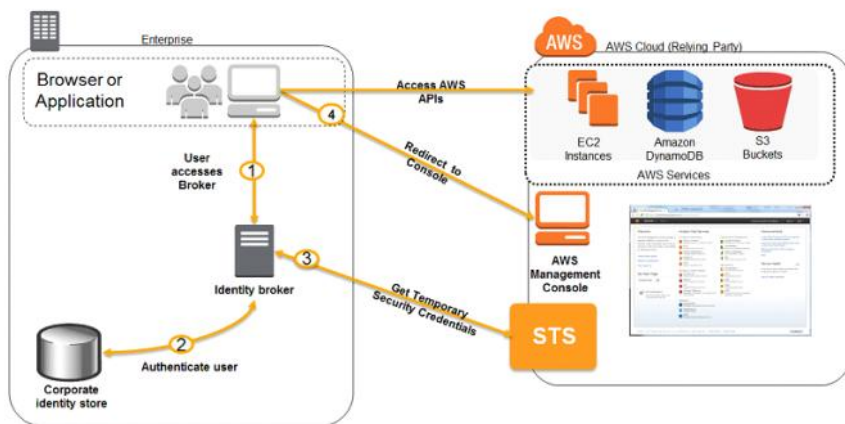
- Implement the AWS IAM Identity Center service to manage access between AWS and your LDAP.
- Set up an IAM policy that references the LDAP identifiers and AWS credentials.
- Create IAM roles to rotate the IAM credentials whenever LDAP credentials are updated.
- Create a custom identity broker application in your on-premises data center and use STS to issue short-lived AWS credentials.

Incorrect

If your identity store is not compatible with SAML 2.0, then you can build a custom identity broker application to perform a similar function. The broker application authenticates users, requests temporary credentials for users from AWS, and then provides them to the user to access AWS resources.

The application verifies that employees are signed into the existing corporate network's identity and authentication system, which might use LDAP, Active Directory, or another system. The identity broker application then obtains temporary security credentials for the employees.

To get temporary security credentials, the identity broker application calls either `AssumeRole` or `GetFederationToken` to obtain temporary security credentials, depending on how you want to manage the policies for users and when the temporary credentials should expire. The call returns temporary security credentials consisting of an AWS access key ID, a secret access key, and a session token. The identity broker application makes these temporary security credentials available to the internal company application. The app can then use the temporary credentials to make calls to AWS directly. The app caches the credentials until they expire, and then requests a new set of temporary credentials.



Hence, the correct answer is: **Create a custom identity broker application in your on-premises data center and use STS to issue short-lived AWS credentials.**

The option that says: **Setting up an IAM policy that references the LDAP identifiers and AWS credentials** is incorrect because using an IAM policy is not enough to integrate your LDAP service into IAM. You need to use SAML, STS, or a custom identity broker instead.

The option that says: **Implementing the AWS IAM Identity Center service to manage access between AWS and your LDAP** is incorrect because the identity store that you are using is not SAML-compatible. AWS IAM Identity Center does not support non-SAML authentication methods.

The option that says: **Creating IAM roles to rotate the IAM credentials whenever LDAP credentials are updated** is incorrect because manually rotating the IAM credentials is not an optimal solution to integrate your on-premises and VPC network. You need to use SAML, STS, or a custom identity broker.

References:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_common-scenarios_federated-users.html

<https://aws.amazon.com/blogs/aws/aws-identity-and-access-management-now-with-identity-federation/>

Check out this AWS Identity & Access Management (IAM) Cheat Sheet:

<https://tutorialsdojo.com/aws-identity-and-access-management-iam/>

18. Question

Category: CDA – Security

A multinational investment bank has a hybrid cloud architecture with AWS. To improve the security of the applications, the company decided to use AWS Key Management Service (KMS) with customer-managed keys to create and manage the encryption keys across a wide range of AWS services. A software developer has been assigned to integrate AWS KMS with the financial applications of the company.

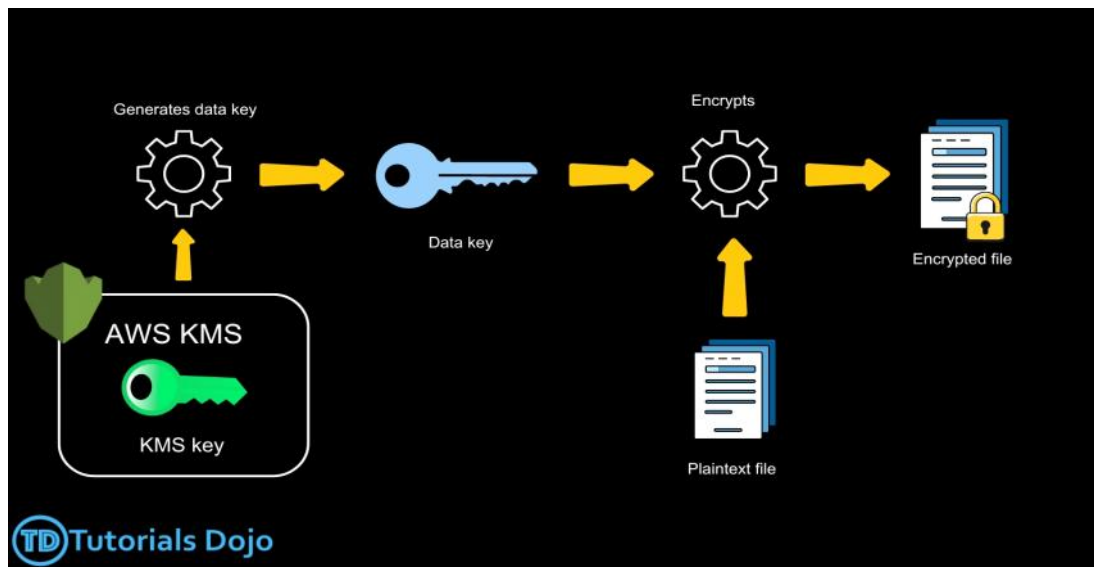
Which of the following are the recommended steps to locally encrypt data using AWS KMS that should be followed? (Select TWO.)

- Erase the encrypted data key from memory and store the plaintext data key alongside the locally encrypted data.
- Encrypt data locally using the `Encrypt` operation.
- Use the `GenerateDataKey` operation to get a data encryption key then use the plaintext data key in the response to encrypt data locally.
- Use the `GenerateDataKeyWithoutPlaintext` operation to get a data encryption key then use the plaintext data key in the response to encrypt data locally.
- Erase the plaintext data key from memory and store the encrypted data key alongside the locally encrypted data.

Incorrect

When you encrypt your data, your data is protected, but you have to protect your encryption key. One strategy is to encrypt it. *Envelope encryption* is the practice of encrypting plaintext data with a data key, and then encrypting the data key under another key.

You can even encrypt the data encryption key under another encryption key, and encrypt that encryption key under another encryption key. But, eventually, one key must remain in plaintext so you can decrypt the keys and your data. This top-level plaintext encryption key is known as the *root key*.



AWS KMS helps you to protect your encryption keys by storing and managing them securely. Root keys stored in AWS KMS, known as AWS KMS keys, never leave the AWS KMS FIPS validated hardware security modules

unencrypted. To use an AWS KMS key, you must call AWS KMS.

It is recommended that you use the following pattern to encrypt data locally in your application:

1. Use the `GenerateDataKey` operation to get a data encryption key.
2. Use the plaintext data key (returned in the `Plaintext` field of the response) to encrypt data locally, then erase the plaintext data key from memory.
3. Store the encrypted data key (returned in the `CiphertextBlob` field of the response) alongside the locally encrypted data.

Hence, the correct answers are:

- Use the `GenerateDataKey` operation to get a data encryption key then use the plaintext data key in the response to encrypt data locally.
- Erase the plaintext data key from memory and store the encrypted data key alongside the locally encrypted data.

The option that says: `Use the GenerateDataKeyWithoutPlaintext` operation to get a data encryption key then using the plaintext data key in the response to encrypt data locally is incorrect because you have to typically use the `GenerateDataKey` operation instead. This is because the `GenerateDataKeyWithoutPlaintext` operation will not return the plaintext data key just as its name implies.

The option that says: `Erase the encrypted data key from memory and storing the plaintext data key alongside the locally encrypted data` is incorrect because it should be the other way around. You have to erase the plaintext data key from memory and store the encrypted data key alongside the locally encrypted data.

The option that says: `Encrypt data locally using the Encrypt` operation is incorrect because the `Encrypt` operation is primarily used to encrypt RSA keys, database passwords, or other sensitive information. This operation can also be used to move encrypted data from one AWS region to another; however, this is not recommended if you want to encrypt your data locally. You have to use the `GenerateDataKey` operation instead.

References:

https://docs.aws.amazon.com/kms/latest/APIReference/API_GenerateDataKey.html
<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#enveloping>

Check out these Amazon S3 and AWS KMS Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>
<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

19. Question

Category: CDA – Security

A product design firm has adopted a remote work policy and wants to provide employees with access to a suite of CAD software through EC2 Spot instances. These instances will be deployed using a CloudFormation template. The development team must be able to securely obtain software license keys in the template each time it is needed.

Which solution meets this requirement while offering the most secure and cost-effective approach?

- Embed the license keys in the `Mapping` section of the CloudFormation template. Let users choose the correct license key using the `Parameter` section. Enable the `NoEcho` attribute on the parameter.
- Store the license key as a secret in AWS Secrets Manager. Use the `secretsmanager` dynamic reference to retrieve the secret in the CloudFormation template.
- Store the license key as a `SecureString` in AWS Systems Manager (SSM) Parameter Store. Use the `ssm-secure` dynamic reference to retrieve the secret in the CloudFormation template.
- Pass the license key in the `Parameter` section of the CloudFormation template during stack creation. Enable the `NoEcho` attribute on the parameter.

Incorrect

Dynamic references provide a compact, powerful way for you to reference external values that are stored and managed in other services, such as the AWS Systems Manager Parameter Store or AWS Secrets Manager. When you use a dynamic reference, CloudFormation retrieves the value of the specified reference when necessary during stack and change set operations and passes the value to the appropriate resource. CloudFormation does not store the actual reference value.

The following snippet shows how you can use the `ssm-secure` dynamic reference to retrieve an IAM user's password from the Parameter Store for console login. `IAMUserPassword` pertains to the parameter name followed by the version number.

MyIAMUser:

```
Type: AWS::IAM::User
Properties:
  UserName: 'MyUserName'
  LoginProfile:
    Password: '{{resolve:ssm-secure:IAMUserPassword:10}}'
```

In the scenario, storing the license key as `SecureString` means encrypting it using a KMS key, making it more secure than storing it in plaintext. It's also more cost-effective than Secrets Manager since you don't pay for the number of parameters you store in the Parameter Store (Standard tier).

Hence, the correct answer is: **Store the license key as a `SecureString` in AWS Systems Manager (SSM) Parameter Store. Use the `ssm-secure` dynamic reference to retrieve the secret in the CloudFormation template.**

The option that says: **Pass the license key in the `Parameter` section of the CloudFormation template during stack creation. Enable the `NoEcho` attribute on the parameter** is incorrect. Although using the `NoEcho` attribute can help prevent the license key from being displayed in plaintext in the CloudFormation logs and console outputs,

The option that says: **Store the license key as a secret in AWS Secrets Manager. Use the `secretsmanager` dynamic reference to retrieve the secret in the CloudFormation template** is incorrect. Although using Secrets Manager is a valid approach, it's less cost-effective compared to using SSM Parameter Store. With Secrets Manager, there is a monthly cost associated with storing secrets, whereas SSM Parameter Store (Standard tier) is free of charge.

The option that says: **Embed the license keys in the `Mapping` section of the CloudFormation template. Let users choose the correct license key using the `Parameter` section. Enable the `NoEcho` attribute on the parameter** is incorrect. Embedding sensitive data, such as license keys, within CloudFormation templates poses a security risk, as the data can be viewed by anyone with access to the template.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/dynamic-references.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/security-best-practices.html#creds>
<https://catalog.workshops.aws/cfn101/en-US/intermediate/templates/dynamic-references>

Check out this AWS CloudFormation cheat sheet:

<https://tutorialsdojo.com/aws-cloudformation/>

20. Question

Category: CDA – Security

A company has different AWS accounts, namely Account A, Account B, and Account C, which are used for their Development, Test, and Production environments respectively. A developer needs access to perform an audit whenever a new version of the application has been deployed to the Test (Account B) and production (Account C) environments.

What is the MOST efficient way to provide the developer access to execute the specified task?

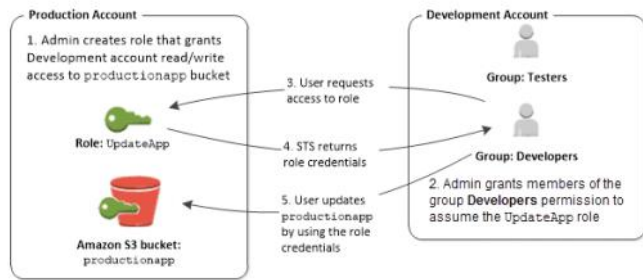
- Grant the developer cross-account access to the resources of Accounts B and C.
- Enable AWS multi-factor authentication (MFA) to the IAM User of the developer.
- Set up AWS Organizations and attach a Service Control Policy to the developer to access the other accounts.
- Create separate identities and passwords for the developer on both the Test and Production accounts.

Incorrect

You can grant your IAM users permission to switch to roles within your AWS account or to roles defined in other AWS accounts that you own.

Imagine that you have Amazon EC2 instances that are critical to your organization. Instead of directly granting your users permission to terminate the instances, you can create a role with those privileges. Then allow administrators to switch to the role when they need to terminate an instance. Doing this adds the following layers of protection to the instances:

- You must explicitly grant your users permission to assume the role.
- Your users must actively switch to the role using the AWS Management Console or assume the role using the AWS CLI or AWS API.
- You can add multi-factor authentication (MFA) protection to the role so that only users who sign in with an MFA device can assume the role.



You can use a role to delegate access to resources that are in different AWS accounts that you own (Production and Testing). You can share resources in one account with users in a different account by setting up cross-account access. In this way, you don't need to create individual IAM users in each account and the users don't have to sign out of one account and sign into another in order to access resources that are in different AWS accounts.

Hence, the most efficient answer in this scenario is to: **Grant the developer cross-account access to the resources of Accounts B and C.**

The option that says: **Create separate identities and passwords for the developer on both the Test and Production accounts** is incorrect because although this is a valid option, it is not an efficient one since the developer will have to log in to each individual AWS account in order to access the resources. A better way to do this is to grant cross-account access.

The option that says: **Enable AWS multi-factor authentication (MFA) to the IAM User of the developer** is incorrect because although this will improve the security, it still doesn't solve the access problem of the developer. This can be used in conjunction with cross-account access, but using this alone will not suffice.

The option that says: **Set up AWS Organizations and attach a Service Control Policy to the developer to access the other accounts** is incorrect because although it will make the multiple AWS accounts of the company more manageable, it doesn't address the access requirement of the developer. Take note that SCPs are necessary, but not sufficient, for granting access for the accounts in your organization. You still need to attach IAM policies to users and roles in your organization's accounts to actually grant permissions to them.

References:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_common-scenarios_aws-accounts.html

<https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>

Check out this AWS Identity & Access Management (IAM) Cheat Sheet:

<https://tutorialsdojo.com/aws-identity-and-access-management-iam/>

21. Question

Category: CDA – Security

A developer is working on an application that will process files encrypted with a data key generated from a KMS key. The application needs to decrypt the files locally before it can proceed with the processing of the files.

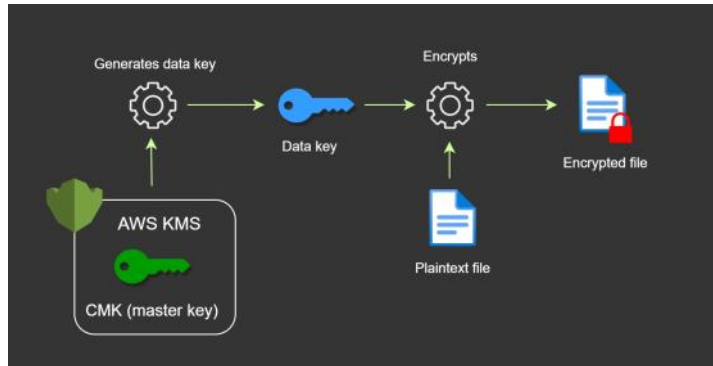
Which of the following are valid and secure steps in decrypting data? (Select TWO.)

- Use the Decrypt operation to decrypt the encrypted data key.
- Use the plaintext data key to decrypt data locally, then erase the encrypted data key from memory.
- Use the Decrypt operation to decrypt the plaintext data key.
- Use the encrypted data key to decrypt data locally, then erase the encrypted data key from memory.
- Use the plaintext data key to decrypt data locally, then erase the plaintext data key from memory.

Incorrect

When you encrypt your data, it is protected, but you have to protect your encryption key. One strategy is to encrypt it. **Envelope encryption** is the practice of encrypting plaintext data with a data key and then encrypting the data key under another key.

You can even encrypt the data encryption key under another encryption key, and encrypt that encryption key under another encryption key. But, eventually, one key must remain in plaintext so you can decrypt the keys and your data. This top-level plaintext key encryption key is known as the root key.



AWS KMS helps you to protect your encryption keys by storing and managing them securely. Root keys stored in AWS KMS, known as AWS KMS keys, never leave the AWS KMS FIPS validated hardware security modules unencrypted. To use an AWS KMS key, you must call AWS KMS.

It is recommended that you use the following pattern to encrypt data locally in your application:

1. Use the `GenerateDataKey` operation to get a data encryption key.
2. Use the plaintext data key (returned in the `Plaintext` field of the response) to encrypt data locally, then erase the plaintext data key from memory.
3. Store the encrypted data key (returned in the `CiphertextBlob` field of the response) alongside the locally encrypted data.

To decrypt data locally:

1. Use the `Decrypt` operation to decrypt the encrypted data key. The operation returns a plaintext copy of the data key.
2. Use the plaintext data key to decrypt data locally, then erase the plaintext data key from memory.

The option that says: **Use the Decrypt operation to decrypt the plaintext data key** is incorrect because there is no need to decrypt a plaintext, or 'unencrypted', data key. The correct way is to use the Decrypt operation to decrypt the encrypted data key.

The option that says: **Use the plaintext data key to decrypt data locally, then erase the encrypted data key from memory** is incorrect. Although this is a valid option for the encryption process, you still must erase the plaintext data key instead of the encrypted one. Take note that you are asked to choose valid and secure steps as per the scenario.

The option that says: **Use the encrypted data key to decrypt data locally, then erase the encrypted data key from memory** is incorrect because you cannot decrypt data using an encrypted data key. You have to decrypt the encrypted data key first and use the plaintext copy of the data key to decrypt the files.

References:

https://docs.aws.amazon.com/kms/latest/APIReference/API_GenerateDataKey.html

<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#enveloping>

Check out these Amazon S3 and AWS KMS Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>

<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

22. Question

Category: CDA – Security

An application hosted in an Auto Scaling group of On-Demand EC2 instances is used to process data polled from an SQS queue, and the generated output is stored in an S3 bucket. To enhance security, you were tasked to ensure that all objects in the S3 bucket are encrypted at rest using server-side encryption with AWS KMS keys.

Which of the following is required to properly implement this requirement?

- Add a bucket policy which denies any `s3:PutObject` action unless the request includes the `x-amz-server-side-encryption-aws-kms-key-id` header.
- Add a bucket policy which denies any `s3:PostObject` action unless the request includes the `x-amz-server-side-encryption` header.

- Add a bucket policy which denies any `s3:PutObject` action unless the request includes the `x-amz-server-side-encryption` header.
- Add a bucket policy which denies any `s3:PostObject` action unless the request includes the `x-amz-server-side-encryption-aws-kms-key-id` header.

Incorrect

Server-side encryption is about protecting data at rest. AWS Key Management Service (AWS KMS) is a service that combines secure, highly available hardware and software to provide a key management system scaled for the cloud. AWS KMS uses KMS keys to encrypt your Amazon S3 objects. You use AWS KMS via the AWS Management Console or AWS KMS APIs to centrally create encryption keys, define the policies that control how keys can be used, and audit key usage to prove they are being used correctly. You can use these keys to protect your data in Amazon S3 buckets.

The first time you add an SSE-KMS-encrypted object to a bucket in a region, a default KMS key is created for you automatically. This key is used for SSE-KMS encryption unless you select a KMS key that you created separately using AWS Key Management Service. Creating your own KMS key gives you more flexibility, including the ability to create, rotate, disable, and define access controls, and to audit the encryption keys used to protect your data.

Default encryption

Automatically encrypt new objects stored in this bucket. [Learn more](#)

Server-side encryption

☐ Disable

☒ Enable

Encryption key type

To upload an object with a customer-provided encryption key (SSE-C), use the AWS CLI, AWS SDK, or Amazon S3 REST API.

☐ Amazon S3 key (SSE-S3)

An encryption key that Amazon S3 creates, manages, and uses for you. [Learn more](#)

☒ AWS Key Management Service key (SSE-KMS)

An encryption key protected by AWS Key Management Service (AWS KMS). [Learn more](#)

AWS KMS key

☒ AWS managed key (aws/s3)

arn:aws:kms:us-east-2:123456789123:alias/aws/s3

☐ Choose from your KMS master keys

☐ Enter KMS master key ARN

Bucket Key

Reduce encryption costs by decreasing calls to AWS KMS for new objects in this bucket. To specify a Bucket Key setting for an object, use the AWS CLI, AWS SDK, or Amazon S3 Rest API. [Learn more](#)

☐ Disable

☒ Enable

Cancel

Save changes

Amazon S3 supports bucket policies that you can use if you require server-side encryption for all objects that are stored in your bucket. For example, you can set a bucket policy that denies permission to upload an object (`s3:PutObject`) to everyone if the request does not include the `x-amz-server-side-encryption` header requesting server-side encryption with SSE-KMS.

When you upload an object, you can specify the KMS key using the `x-amz-server-side-encryption-aws-kms-key-id` header which you can use to require a specific KMS key for object encryption. If the header is not present in the request, Amazon S3 assumes the default KMS key. Regardless, the KMS key ID that Amazon S3 uses for object encryption must match the KMS key ID in the policy, otherwise Amazon S3 denies the request.

Therefore, the correct answer is: **Add a bucket policy which denies any `s3:PutObject` action unless the request includes the `x-amz-server-side-encryption-aws-kms-key-id` header.**

The option that says: **Add a bucket policy which denies any `s3:PutObject` action unless the request includes the `x-amz-server-side-encryption` header** is incorrect. While this policy ensures that objects are encrypted, it does not enforce the use of AWS KMS keys. The `x-amz-server-side-encryption` header can typically specify different encryption methods, including SSE-S3 (Amazon S3 managed keys) or SSE-KMS (AWS KMS keys). Since the requirement is to use AWS KMS keys specifically, this option is not sufficient.

The option that says: **Add a bucket policy which denies any `s3:PostObject` action unless the request includes the `x-amz-server-side-encryption` header** is incorrect because the correct action for uploading objects is `s3:PutObject`, not the `s3:PostObject` action. Additionally, it does not specifically enforce the use of AWS KMS keys.

The option that says: **Adding a bucket policy which denies any `s3:PostObject` action unless the request includes the `x-amz-server-side-encryption-aws-kms-key-id` header** is incorrect because it uses the `s3:PostObject` action instead of the correct `s3:PutObject` action. Even though it primarily uses the AWS KMS keys, the incorrect action makes this policy ineffective for ensuring encryption during object uploads.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/serv-side-encryption.html>

<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingKMSEncryption.html>

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/specifying-kms-encryption.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdoido.com/amazon-s3/>

23. Question

Category: CDA – Security

A developer needs to encrypt all objects being uploaded by their application to the S3 bucket to comply with the company's security policy. The bucket will use server-side encryption with Amazon S3-Managed encryption keys (SSE-S3) to encrypt the data using 256-bit Advanced Encryption Standard (AES-256) block cipher.

Which of the following request headers should the developer use?

- `x-amz-server-side-encryption-customer-algorithm`
- `x-amz-server-side-encryption-customer-key`
- `x-amz-server-side-encryption`
- `x-amz-server-side-encryption-customer-key-MD5`

Incorrect

Server-side encryption protects data at rest. If you use Server-Side Encryption with Amazon S3-Managed Encryption Keys (SSE-S3), Amazon S3 will encrypt each object with a unique key and as an additional safeguard, it encrypts the key itself with a master key that it rotates regularly. Amazon S3 server-side encryption uses one of the strongest block ciphers available, 256-bit Advanced Encryption Standard (AES-256), to encrypt your data.

Edit default encryption [Info](#)

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [Info](#)

- ☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)
- ☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)
- ☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)
Secure your objects with two separate layers of encryption. For details on pricing, see [DSSE-KMS pricing](#) on the **Storage** tab of the [Amazon S3 pricing page](#).

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

- ☐ Disable
- ☒ Enable



Cancel

Save changes

If you need server-side encryption for all of the objects that are stored in a bucket, use a bucket policy. For example, the following bucket policy denies permissions to upload an object unless the request includes the `x-amz-server-side-encryption` header to request server-side encryption:

```
{
  "Version": "2012-10-17",
  "Id": "PutObjectPolicy",
  "Statement": [
    {
      "Sid": "DenyIncorrectEncryptionHeader",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::awsexamplebucket1/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    }
  ]
}
```

However, if you chose to use server-side encryption with customer-provided encryption keys (SSE-C), you must provide encryption key information using the following request headers:

`x-amz-server-side-encryption-customer-algorithm`
`x-amz-server-side-encryption-customer-key`
`x-amz-server-side-encryption-customer-key-MD5`

Hence, using the `x-amz-server-side-encryption` header is correct as this is the one being used for Amazon S3-Managed Encryption Keys (SSE-S3). All other options are incorrect since they are used for SSE-C.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/serv-side-encryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingServerSideEncryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/dev/ServerSideEncryptionCustomerKeys.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

24. Question

Category: CDA – Security

A company has a static website running in an Auto Scaling group of Amazon EC2 instances, which should be converted into a dynamic e-commerce web portal. One of the requirements is to use HTTPS to improve the security of the portal and also improve its search ranking as a reputable and secure site. A developer recently requested an SSL/TLS certificate from a third-party certificate authority (CA) which is ready to be imported to AWS. The company is also evaluating the use of AWS Private Certificate Authority to issue internal certificates for other workloads.

Which of the following services can the developer use to safely import the SSL/TLS certificate? (Select TWO.)

- A private Amazon S3 bucket with versioning enabled
- Amazon CloudFront
- IAM certificate store
- AWS Certificate Manager

- Amazon Cognito

Incorrect

To enable HTTPS connections to your website or application in AWS, you need an **SSL/TLS server certificate**. For certificates in a Region supported by AWS Certificate Manager (ACM), it is recommended that you use ACM to provision, manage, and deploy your server certificates. In unsupported Regions, you must use IAM as a certificate manager.

ACM is the preferred tool to provision, manage, and deploy your server certificates. With ACM you can request a certificate or deploy an existing ACM or external certificate to AWS resources. Certificates provided by ACM are free and automatically renew. In a supported Region, you can use ACM to manage server certificates from the console or programmatically. Use IAM as a certificate manager only when you must support HTTPS connections in a Region that is not supported by ACM. IAM securely encrypts your private keys and stores the encrypted version in IAM SSL certificate storage. IAM supports deploying server certificates in all Regions, but you must obtain your certificate from an external provider for use with AWS. You cannot upload an ACM certificate to IAM. Additionally, you cannot manage your certificates from the IAM Console. If you got your certificate from a third-party CA, import the certificate into ACM or upload it to the IAM certificate store.

- Hence, the correct answers are:
- **AWS Certificate Manager (ACM)**
 - **IAM certificate store.**

A private Amazon S3 bucket with versioning enabled is incorrect as S3 is not a suitable service to store the SSL certificate. You have to import it to either AWS Certificate Manager (ACM) or IAM certificate store. **Amazon Cognito** is incorrect because this is just a user identity and data synchronization service that helps you securely manage and synchronize app data for your users across their mobile devices. This service can't be used to store or import your SSL certificates. **Amazon CloudFront** is incorrect. Although you can upload certificates to CloudFront, it doesn't mean that you can import third-party SSL certificates on it. If you got your certificate from a third-party CA then you typically have to import the certificate into ACM or upload it to the IAM certificate store first. You would also not be able to export the certificate that you have loaded in CloudFront nor assign them to your EC2 or ELB instances as it would be tied to a single CloudFront distribution.

References:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_server-certs.html
<https://aws.amazon.com/certificate-manager/>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cnames-and-https-procedures.html#cnames-and-https-uploading-certificates>

Check out this AWS Certificate Manager Cheat Sheet:
<https://tutorialsdojo.com/aws-certificate-manager/>

25. 25. Question

Category: CDA – Security

A company has created a private S3 bucket named `tdojo`. The Developer IAM role must be granted read access to all objects within this bucket. However, the QA IAM role should be restricted to accessing only the objects stored under the `qa` folder.

Which S3 bucket policy will effectively implement the principle of least privilege access while satisfying the given requirements?

- {


```

      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "AWS": [
              "arn:aws:iam::123456789123:role/Developer"
            ]
          },
          "Action": [
            "s3:*"
          ],
          "Resource": "arn:aws:s3:::tdojo/*"
        },
        {
          "Effect": "Allow",
          "Principal": {
            "AWS": [
              "arn:aws:iam::123456789123:role/QA"
            ]
          },
          "Action": [
            "s3:GetObject"
          ],
          "Resource": "arn:aws:s3:::tdojo/qa/*"
        }
      ]
    }
      
```
- {


```

      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "AWS": [
              "arn:aws:iam::123456789123:role/Developer"
            ]
          },
          "Action": [
            "s3:GetObject"
          ],
          "Resource": "arn:aws:s3:::tdojo/*"
        }
      ]
    }
      
```

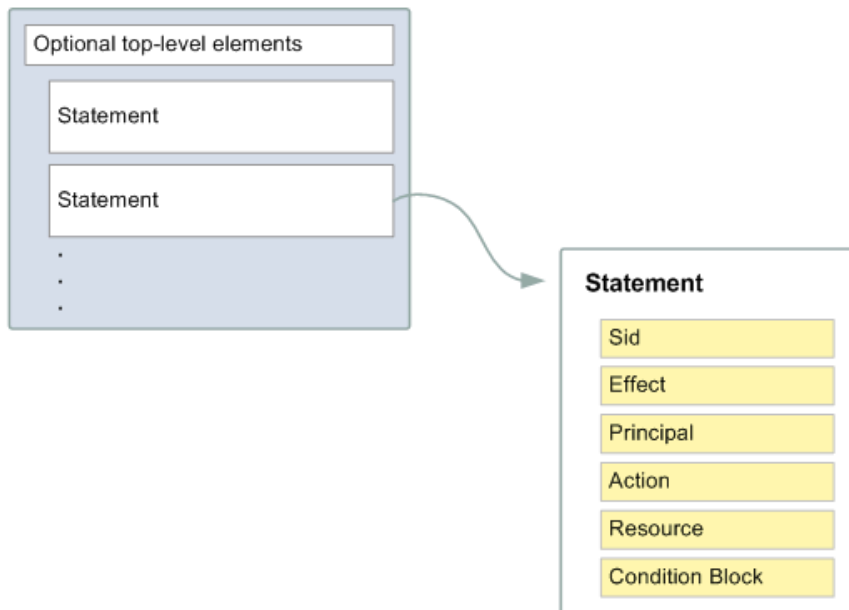
```

    "Resource": "arn:aws:s3:::tdojo/*"
  },
  {
    "Effect": "Allow",
    "Principal": {
      "AWS": [
        "arn:aws:iam::123456789123:role/QA"
      ]
    },
    "Action": [
      "s3:GetObject"
    ],
    "Resource": "arn:aws:s3:::tdojo/qa/*"
  }
}
• {
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/Developer",
          "arn:aws:iam::123456789123:role/QA"
        ]
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::tdojo/*",
        "arn:aws:s3:::tdojo/qa/*"
      ]
    }
  ]
}
• {
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/Developer"
        ]
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::tdojo/*"
    },
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/QA"
        ]
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::tdojo/qa/*"
    }
  ]
}

```

Incorrect

An S3 bucket policy is a JSON document that defines access controls for an Amazon S3 bucket. It allows you to grant or deny access to different AWS identities (such as IAM users and roles or federated users) for different S3 bucket actions (such as GetObject, PutObject, and ListBucket) on specific S3 objects or prefixes within a bucket. It can be used to control who can access your S3 data and how they can access it.



An S3 bucket policy statement is composed of several elements, and the following are required to create a valid policy:

- **Effect:** The effect can be **Allow** or **Deny**.
- **Action:** The specific API action for which you are granting or denying permission.
- **Principal:** The user, account, service, or other entity that is allowed or denied access to the bucket or objects within the bucket.
- **Resource:** The resource that's affected by the action. You specify a resource using an Amazon Resource Name (ARN).

In the scenario, you can create separate statements for the Developer and QA roles. This allows for granular control over access to the bucket and its contents. Each statement can have its own set of conditions, allowing for different permissions to be granted to different identities or groups under different circumstances. Additionally, separating statements can make it easier to read, understand and manage the policy as it becomes more complex.

Hence, the correct answer is:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/Developer"
        ]
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::tdojo/*"
    },
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/QA"
        ]
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::tdojo/qa/*"
    }
  ]
}
  
```

The policy below is not compliant with the principle of least privilege, as it uses a wildcard in the Action element. The scenario mentioned that the Developer IAM role must be given permission to read all objects in the bucket. Therefore, you have to explicitly mention the specific API to allow that operation, which is GetObject.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/Developer"
        ]
      },
      "Action": [
        "s3:*"
      ],
      "Resource": "arn:aws:s3:::tdojo/*"
    },
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/QA"
        ]
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::tdojo/qa/*"
    }
  ]
}
  
```

The policy below does not follow the principle of least privilege since it grants excessive permissions to the Developer IAM role. Note that the Developer IAM role only needs read access.

```

{
  "Version": "2012-10-17",
  "Statement": [
  
```

```

{
  "Effect": "Allow",
  "Principal": {
    "AWS": [
      "arn:aws:iam::123456789123:role/Developer"
    ]
  },
  "Action": [
    "s3:GetObject",
    "s3:PutObject"
  ],
  "Resource": "arn:aws:s3:::tdojo/*"
},
{
  "Effect": "Allow",
  "Principal": {
    "AWS": [
      "arn:aws:iam::123456789123:role/QA"
    ]
  },
  "Action": [
    "s3:GetObject"
  ],
  "Resource": "arn:aws:s3:::tdojo/qa/*"
}
]
}

```

The IAM Policy shown below is not right as it grants the QA role access to read all objects within the S3 bucket, which is typically beyond its necessary permissions:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::123456789123:role/Developer",
          "arn:aws:iam::123456789123:role/QA"
        ]
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::tdojo/*",
        "arn:aws:s3:::tdojo/qa/*"
      ]
    }
  ]
}

```

References:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-iam-policies.html>
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

26. Question

Category: CDA – Security

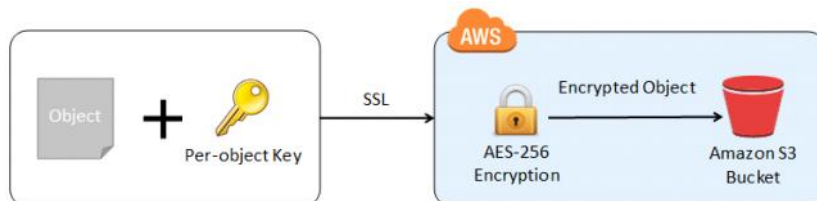
A developer is designing a multi-tiered system which utilizes various AWS resources. The application will be hosted in Elastic Beanstalk, which uses an RDS database and an S3 bucket that is configured to use Server-Side Encryption with Customer-Provided Encryption Keys (SSE-C). In this configuration, Amazon S3 does not store the encryption key you provide but instead, stores a randomly salted hash-based message authentication code (HMAC) value of the encryption key in order to validate future requests.

Which of the following is a valid consideration that the developer should keep in mind when implementing this architecture?

- If you lose the encryption key, you lose the object.
- The salted HMAC value can be used to derive the value of the encryption key.
- If you lose the encryption key, the salted HMAC value can be used to decrypt the object.
- The salted HMAC value can be used to decrypt the contents of the encrypted object.

Incorrect

Server-side encryption is about protecting data at rest. Using server-side encryption with customer-provided encryption keys (SSE-C) allows you to set your own encryption keys. With the encryption key you provide as part of your request, Amazon S3 manages both the encryption, as it writes to disks, and decryption, when you access your objects. Therefore, you don't need to maintain any code to perform data encryption and decryption. The only thing you do is manage the encryption keys you provide.



When you upload an object, Amazon S3 uses the encryption key you provide to apply AES-256 encryption to your data and removes the encryption key from memory. It is important to note that Amazon S3 does not store the encryption key you provide. Instead, it is stored in a randomly salted HMAC value of the encryption key in order to validate future requests. The salted HMAC value cannot be used to derive the value of the encryption key or to decrypt the contents of the encrypted object. That means, if you lose the encryption key, you lose the object.

When you retrieve an object, you must provide the same encryption key as part of your request. Amazon S3 first verifies that the encryption key you provided matches, and then decrypts the object before returning the object data to you.

Hence, the valid consideration that the developer should keep in mind when implementing this architecture is: **if you lose the encryption key, you lose the object.**

The option that says: **the salted HMAC value can be used to derive the value of the encryption key** is incorrect because the salted HMAC is just used to validate future encryption requests. It cannot be used to derive the value of the encryption key or to decrypt the contents of the encrypted object.

The option that says: **the salted HMAC value can be used to decrypt the contents of the encrypted object** is incorrect because just as mentioned above, the HMAC cannot be used to derive the value of the encryption key or to decrypt the contents of the encrypted object.

The option that says: **if you lose the encryption key, the salted HMAC value can be used to decrypt the object** is incorrect because if you lose the encryption key, you lose the object. You cannot use the salted HMAC value to decrypt the object.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingKMSEncryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/API/RESTObjectPUT.html#RESTObjectPUT-responses-examples>
<https://aws.amazon.com/blogs/security/how-to-prevent-uploads-of-unencrypted-objects-to-amazon-s3/>

Check out these Amazon S3 and AWS KMS Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>

27. Question

Category: CDA – Security

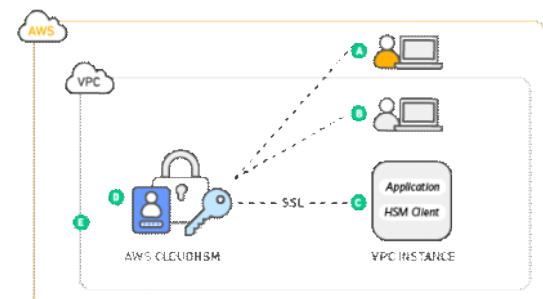
A cryptocurrency exchange portal has a key management service hosted in their on-premises data center, which stores encryption keys and uses an RSA asymmetric encryption algorithm. The company has recently implemented a hybrid cloud architecture in AWS and you were assigned to migrate the exchange portal to their cloud infrastructure. For security compliance, the keys should be stored in dedicated, third-party validated hardware security modules under your exclusive control.

Which of the following is the BEST solution that you should implement to meet the above requirement?

- Import the encryption keys from your on-premises key management service to AWS Secrets Manager as KMS Keys.
- Use AWS KMS to store and manage the encryption keys.
- Develop a custom key management service using the AWS Encryption SDK.
- Import the encryption keys from your on-premises key management service to AWS CloudHSM.

Incorrect

AWS CloudHSM provides hardware security modules in AWS Cloud. A hardware security module (HSM) is a computing device that processes cryptographic operations and provides secure storage for cryptographic keys.



When you use an HSM from AWS CloudHSM, you can perform a variety of cryptographic tasks:

- Generate, store, import, export, and manage cryptographic keys, including symmetric keys and asymmetric key pairs.
- Use symmetric and asymmetric algorithms to encrypt and decrypt data.
- Use cryptographic hash functions to compute message digests and hash-based message authentication codes (HMACs).
- Cryptographically sign data (including code signing) and verify signatures.
- Generate cryptographically secure random data.

You should consider using AWS CloudHSM instead of AWS KMS if you require:

- Keys stored in dedicated, third-party validated hardware security modules under your exclusive control.
- FIPS 140-2 compliance.
- Integration with applications using PKCS#11, Java JCE, or Microsoft CNG interfaces.
- High-performance in-VPC cryptographic acceleration (bulk crypto).

Hence, the correct answer is to **import the encryption keys from your on-premises key management service to AWS CloudHSM**.

Using AWS KMS to store and manage the encryption keys is incorrect. Although AWS KMS supports asymmetric encryption, it doesn't provide dedicated, third-party validated hardware security modules which are under your exclusive control. You have to use CloudHSM instead.

Importing the encryption keys from your on-premises key management service to AWS Secrets Manager as KMS Keys is incorrect because you can't store KMS keys to AWS Secrets Manager.

Developing a custom key management service using the AWS Encryption SDK is incorrect because this entails a lot of effort to implement. Moreover, the AWS Encryption SDK only encrypts your data using a symmetric key algorithm which doesn't comply with the requirements provided in the scenario.

References:

<https://docs.aws.amazon.com/cloudhsm/latest/userguide/manage-keys.html>

<https://aws.amazon.com/cloudhsm/faqs/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

28. Question

Category: CDA – Security

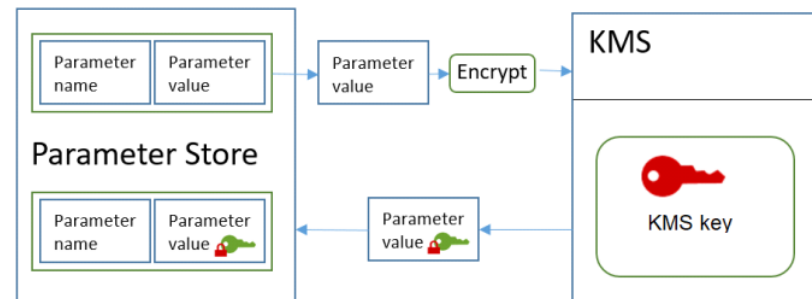
A serverless application is composed of several Lambda functions which reads data from RDS. These functions must share the same connection string that should be encrypted to improve data security.

Which of the following is the MOST secure way to meet the above requirement?

- Create a Secure String Parameter using the AWS Systems Manager Parameter Store.
- Create an IAM Execution Role that has access to RDS and attach it to the Lambda functions.
- Use AWS Lambda environment variables encrypted with KMS which will be shared by the Lambda functions.
- Use AWS Lambda environment variables encrypted with CloudHSM.

Incorrect

AWS Systems Manager Parameter Store provides secure, hierarchical storage for configuration data management and secrets management. You can store data such as passwords, database strings, and license codes as parameter values. You can store values as plain text or encrypted data. You can then reference values by using the unique name that you specified when you created the parameter.



Parameter Store offers the following benefits and features:

- Use a secure, scalable, hosted secrets management service (No servers to manage).
- Improve your security posture by separating your data from your code.
- Store configuration data and secure strings in hierarchies and track versions.
- Control and audit access at granular levels.
- Configure change notifications and trigger automated actions.
- Tag parameters individually, and then secure access from different levels, including operational, parameter, Amazon EC2 tag, or path levels.
- Reference AWS Secrets Manager secrets by using Parameter Store parameters.

Hence, **creating a Secure String Parameter using the AWS Systems Manager Parameter Store** is the correct solution for this scenario.

The option that says: **Use AWS Lambda environment variables encrypted with KMS which will be shared by the Lambda functions** is incorrect. Even though the credentials will be encrypted, these environment variables will only be used by an individual Lambda function, and cannot be shared.

The option that says: **Create an IAM Execution Role that has access to RDS and attach it to the Lambda functions** is incorrect because this solution will not encrypt the database credentials for RDS.

The option that says: **Use AWS Lambda environment variables encrypted with CloudHSM** is incorrect because Lambda primarily uses KMS for encryption and not CloudHSM.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-paramstore.html>
<https://aws.amazon.com/blogs/compute/sharing-secrets-with-aws-lambda-using-aws-systems-manager-parameter-store/>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-systems-manager/>

29. Question

Category: CDA – Security

A web application is currently using an on-premises Microsoft SQL Server 2019 Enterprise Edition database. Your manager instructed you to migrate the application to Elastic Beanstalk and the database to RDS for SQL Server. For additional security, you must configure your database to automatically encrypt the actual data before it is written to storage, and automatically decrypt data when the data is read from storage.

Which of the following services will you use to achieve this?

- Use IAM DB Authentication.
- Enable Transparent Data Encryption (TDE).
- Use Microsoft SQL Server Windows Authentication.
- Enable RDS Encryption.

Incorrect

Amazon RDS supports using Transparent Data Encryption (TDE) to encrypt stored data on your DB instances running Microsoft SQL Server. TDE automatically encrypts data before it is written to storage, and automatically decrypts data when the data is read from storage.

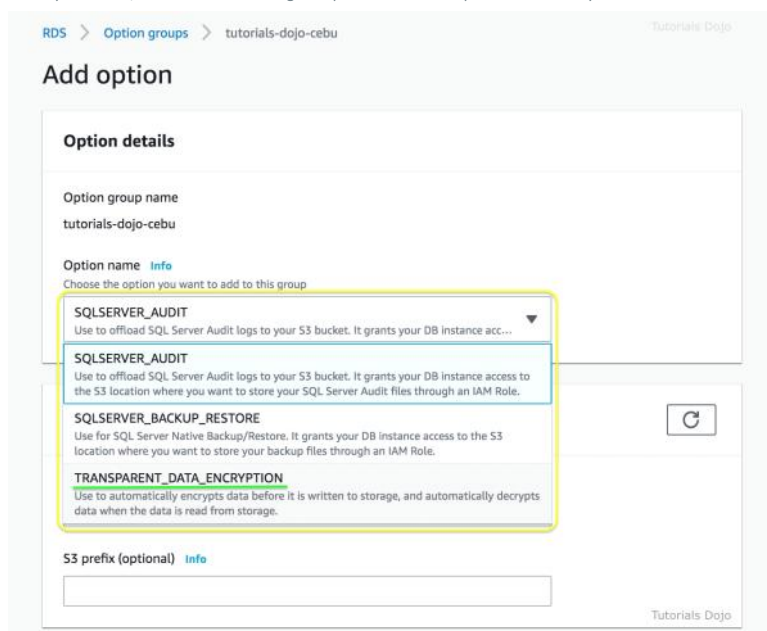
Amazon RDS supports TDE for the following SQL Server versions and editions:

- SQL Server 2019 Standard and Enterprise Editions
- SQL Server 2017 Enterprise Edition
- SQL Server 2016 Enterprise Edition
- SQL Server 2014 Enterprise Edition
- SQL Server 2012 Enterprise Edition

Transparent Data Encryption is used in scenarios where you need to encrypt sensitive data. For example, you might want to provide data files and backups to a third party, or address security-related regulatory compliance issues.

To enable transparent data encryption for an RDS SQL Server DB instance, specify the TDE option in an RDS option group that is associated with that DB instance.

Transparent data encryption for SQL Server provides encryption key management by using a two-tier key architecture. A certificate, which is generated from the database master key, is used to protect the data encryption keys. The database encryption key performs the actual encryption and decryption of data on the user database. Amazon RDS backs up and manages the database master key and the TDE certificate. To comply with several security standards, Amazon RDS is working to implement automatic periodic master key rotation.



TDE encrypts the actual data and log files at the database level. It automatically encrypts data before it is written to storage and automatically decrypts data when it is read from storage. Lastly, TDE provides encryption at the data level, protecting the data itself from unauthorized access.

Hence, the correct answer is to **Enable Transparent Data Encryption (TDE)**.

The option that says: **Use IAM DB Authentication** is incorrect because this option just lets you authenticate to your DB instance using AWS Identity and Access Management (IAM) database authentication. The more appropriate security feature to use here is TDE.

The option that says: **Enable RDS Encryption** is incorrect. This option enables encryption at rest for the underlying storage volumes, but it does not automatically encrypt and decrypt the data itself. It relies on the database engine (in this case, SQL Server) to handle the encryption and decryption of the actual data.

The option that says: **Use Microsoft SQL Server Windows Authentication** is incorrect because this option is primarily used if you want to integrate RDS with your AWS Directory Service for Microsoft Active Directory (also called AWS Managed Microsoft AD) to enable Windows Authentication to authenticate users.

References:

<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Appendix.SQLServer.Options.TDE.html>
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_SQLServer.html

Check out this Amazon RDS Cheat Sheet:

<https://tutorialsdojo.com/amazon-relational-database-service-amazon-rds/>

30. Question

Category: CDA – Security

A developer is using API Gateway Lambda Authorizer to securely authenticate the API requests to their web application. The authentication process should be implemented using a custom authorization scheme which accepts header and query string parameters from the API caller.

Which of the following methods should the developer use to properly implement the above requirement?

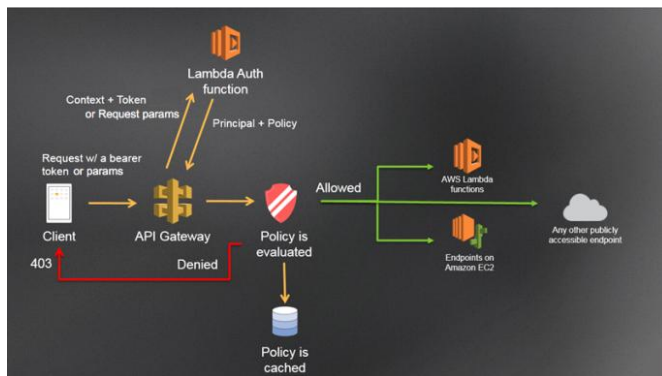
- Cross-Account Lambda Authorizer
- Amazon Cognito User Pools Authorizer
- Request Parameter-based Authorization
- Token-based Authorization

Incorrect

A Lambda authorizer is an API Gateway feature that uses a Lambda function to control access to your API. When a client makes a request to one of your API's methods, API Gateway calls your Lambda authorizer, which takes the caller's identity as input and returns an IAM policy as output.

There are two types of Lambda authorizers:

- A **token-based** Lambda authorizer (also called a TOKEN authorizer) receives the caller's identity in a bearer token, such as a JSON Web Token (JWT) or an OAuth token.
- A **request parameter-based** Lambda authorizer (also called a REQUEST authorizer) receives the caller's identity in a combination of headers, query string parameters, stageVariables, and \$context variables.



It is possible to use an AWS Lambda function from an AWS account that is different from the one in which you created your Lambda authorizer function, by using a Cross-Account Lambda Authorizer. Therefore, the correct answer is to use a **Request Parameter-based Authorization** as it uses a combination of headers, query string parameters, stageVariables, and \$context variables for authentication. **Amazon Cognito User Pools Authorizer** is incorrect because this is just an alternative to using IAM roles and policies or Lambda authorizers. An Amazon Cognito user pool is primarily used to control who can access your API in Amazon API Gateway using identity token or access token, and not header and query string parameters from the API caller. **Token-based authorization** is incorrect because this is only suitable if you want to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML. **Cross-Account Lambda Authorizer** is incorrect because this just enables you to use an AWS Lambda function from a different AWS account as your API authorizer function. Moreover, this is not a valid Lambda authorizer type.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-lambda-authorizer-input.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-lambda-authorizer-cross-account-lambda-authorizer.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

From <<https://portal.tutorialsdojo.com/courses/aws-certified-developer-associate-practice-exams/lessons/practice-exams-section-based-3/quizzes/aws-certified-developer-associate-practice-exam-section-based-security/>>